Vietnam National University of HCMC

International University

School of Computer Science and Engineering

Program of MIT

Course: Advanced Computer Graphics (IT516)

Lecturer: Assoc. Prof. Nguyen Van Sinh

Email: nvsinh@hcmiu.edu.vn

View Transformation

Textbook: chapter 8 (24/3/2026)

Viewing and Projection

- Our eyes collapse 3D world to 2D retinal image

(brain then has to reconstruct 3D)

- In CG, this process occurs by projection
- Projection has two parts:
  - Viewing transformations: camera position and direction
  - Perspective/orthographic transformation: reduces 3D to 2D
- Other names:
  - Camera space is also "eye space"
  - Camera transformation is sometimes the "viewing transformation"
  - Canonical view volume is also "clip space" or "normalized device coordinates"
  - Screen space is also "pixel coordinates."

Recall: homogeneous coordinates

An infinite number of points correspond to $(x,y,1)$.

They constitute the whole line (tx,ty,t).

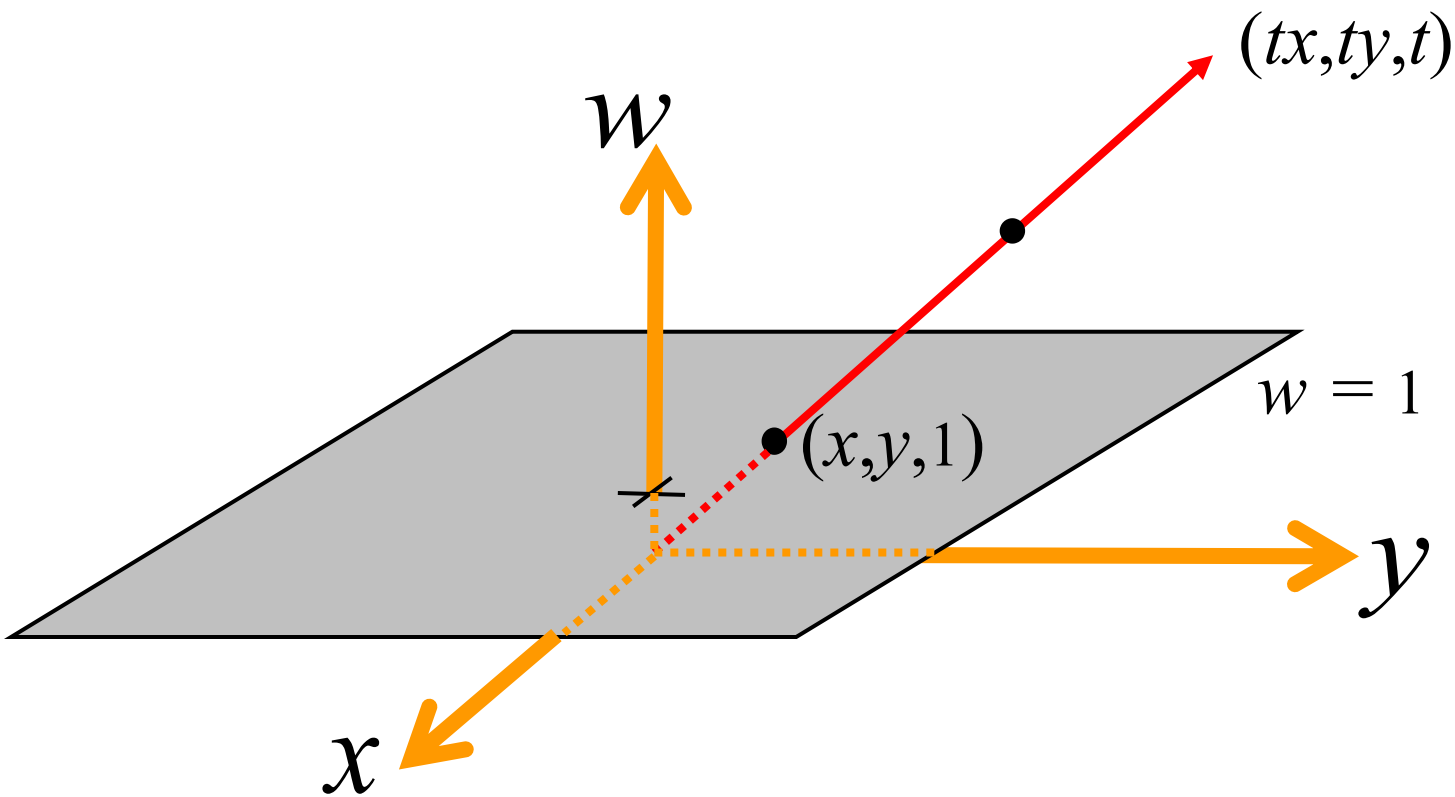


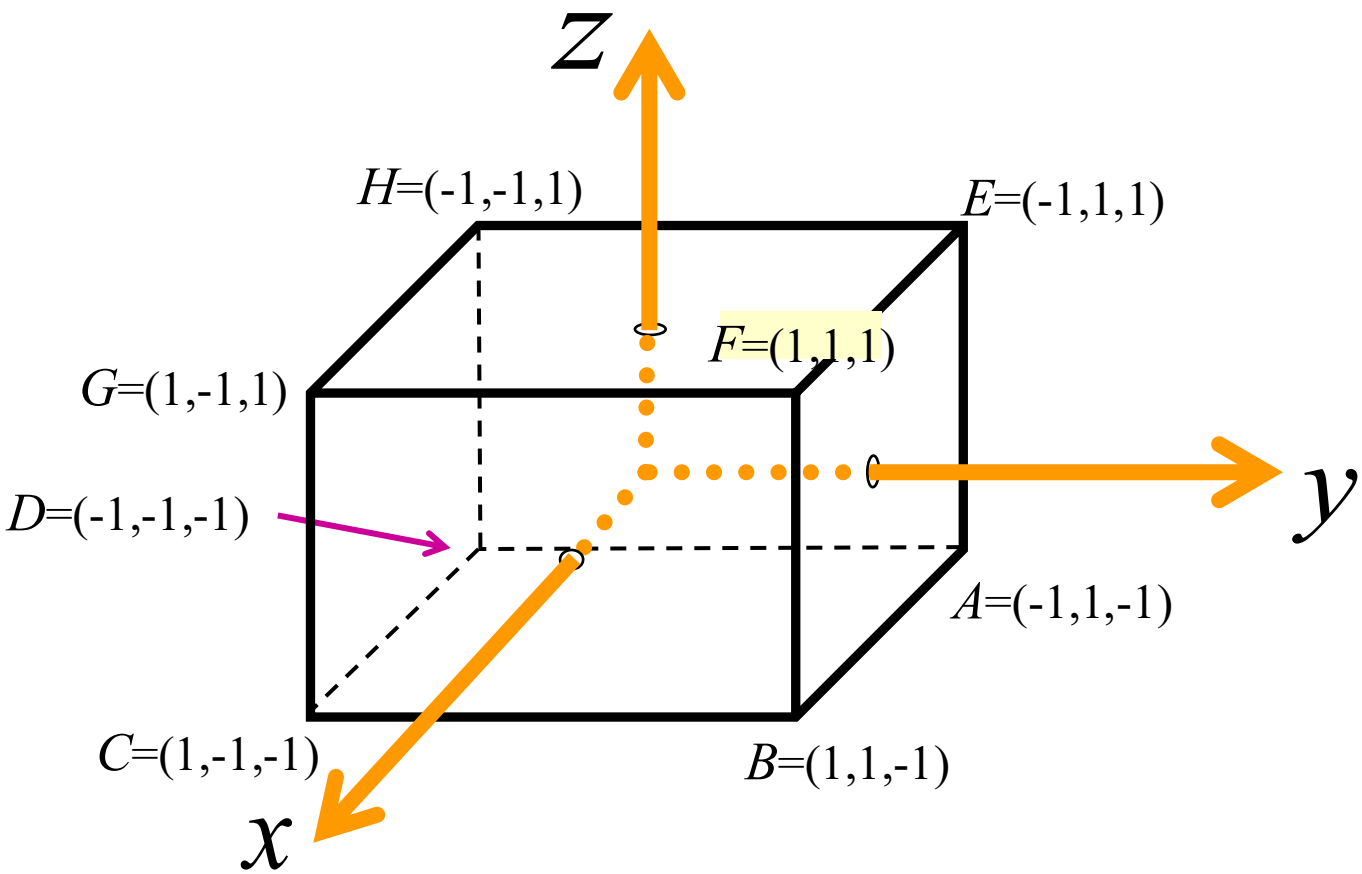
Illustration (Old Approach) :

Simple Transformation Sequence for 3D Viewing

Simple Viewing Transformation Example

Pts	A	B	C	D	E	F	G	H
X	-1	1	1	-1	-1	1	1	-1
Y	1	1	-1	-1	1	1	-1	-1
Z	-1	-1	-1	-1	1	1	1	1

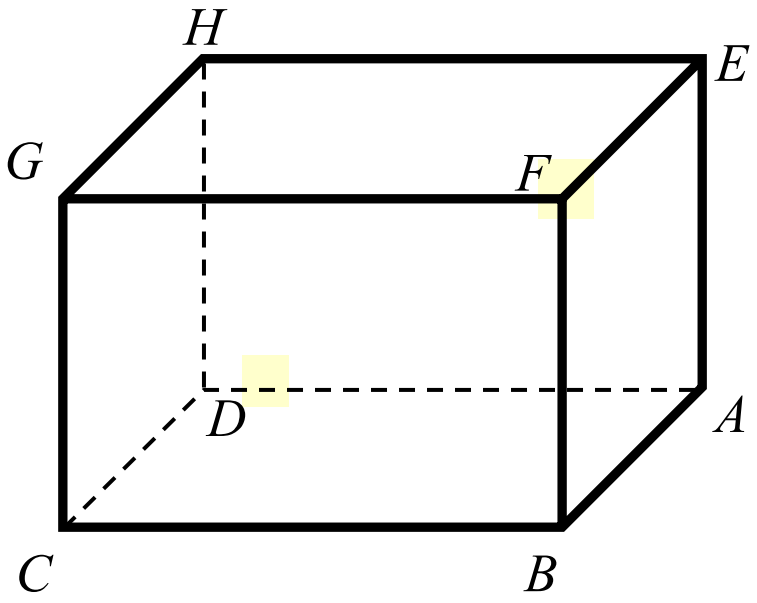
Cube Viewed from (6,8,7.5)



The canonical view volume is a cube with the root coordinate at the center

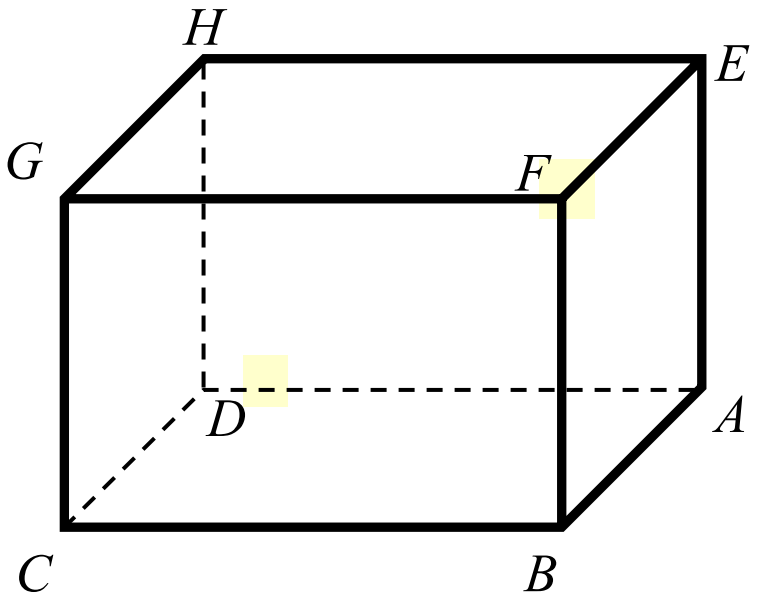
Topology of Cube

	A	*B*	*C*	*D*	*E*	*F*	*G*	*H*
A	0	1	0	1	1	0	0	0
B	1	0	1	0	0	1	0	0
C	0	1	0	1	0	0	1	0
D	1	0	1	0	0	0	0	1
E	1	0	0	0	0	1	0	1
F	0	1	0	0	1	0	1	0
G	0	0	1	0	0	1	0	1
H	0	0	0	1	1	0	1	0



Compressed Cube *Topology*

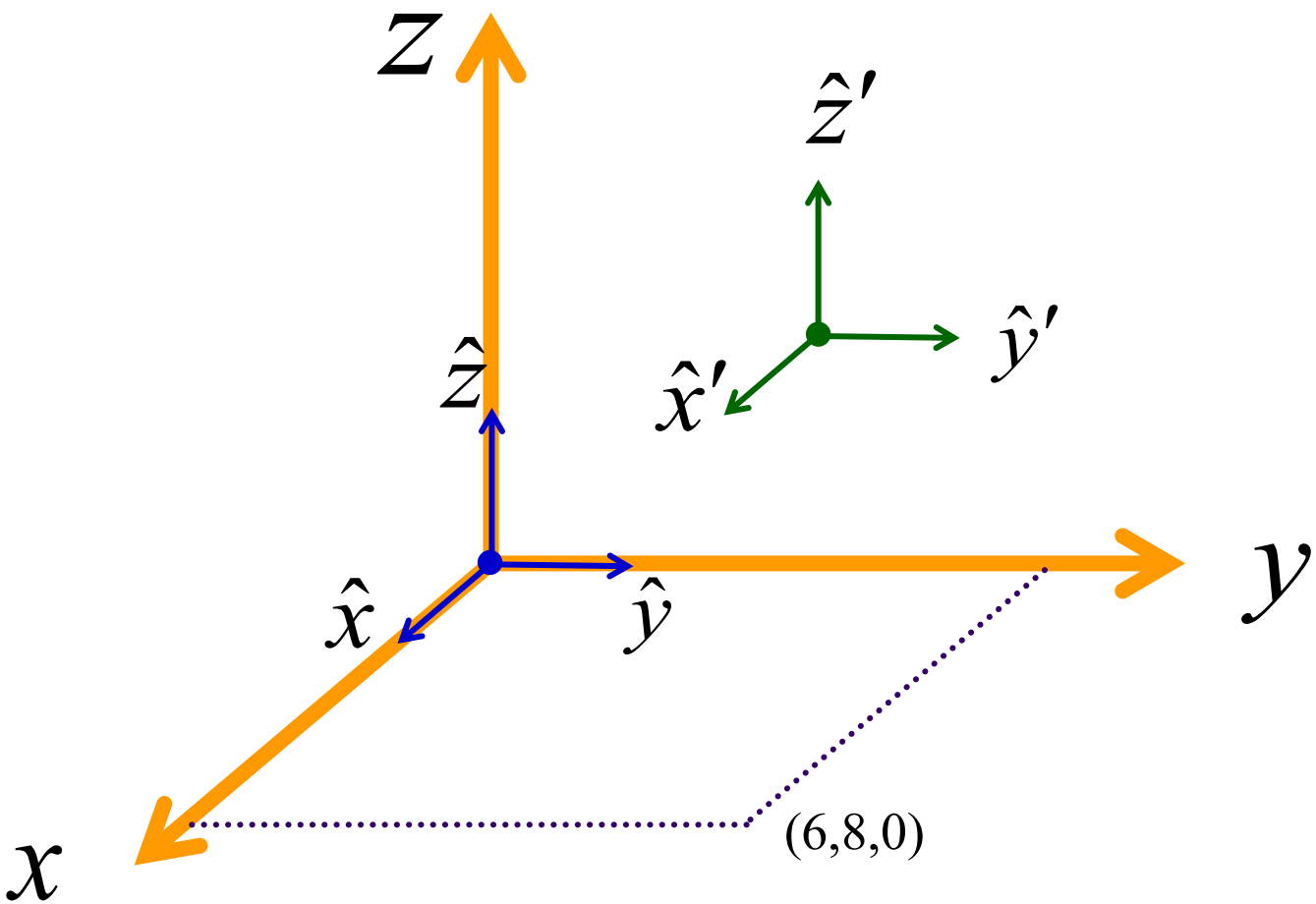
A:	*B*	*D*	*E*
B:	*A*	*C*	*F*
C:	*B*	*D*	*G*
D:	*A*	*C*	*H*
E:	*A*	*F*	*H*
F:	*B*	*E*	*G*
G:	*C*	*F*	*H*
H:	*D*	*E*	*G*



Simple Example

- Given Cube with Corners
- View from *Eye Position* $(6,8,7.5)$ $(\pm 1, \pm 1, \pm 1)$
- Look at *Origin* $(0,0,0)$
- $z\text{-}direction$ is Up

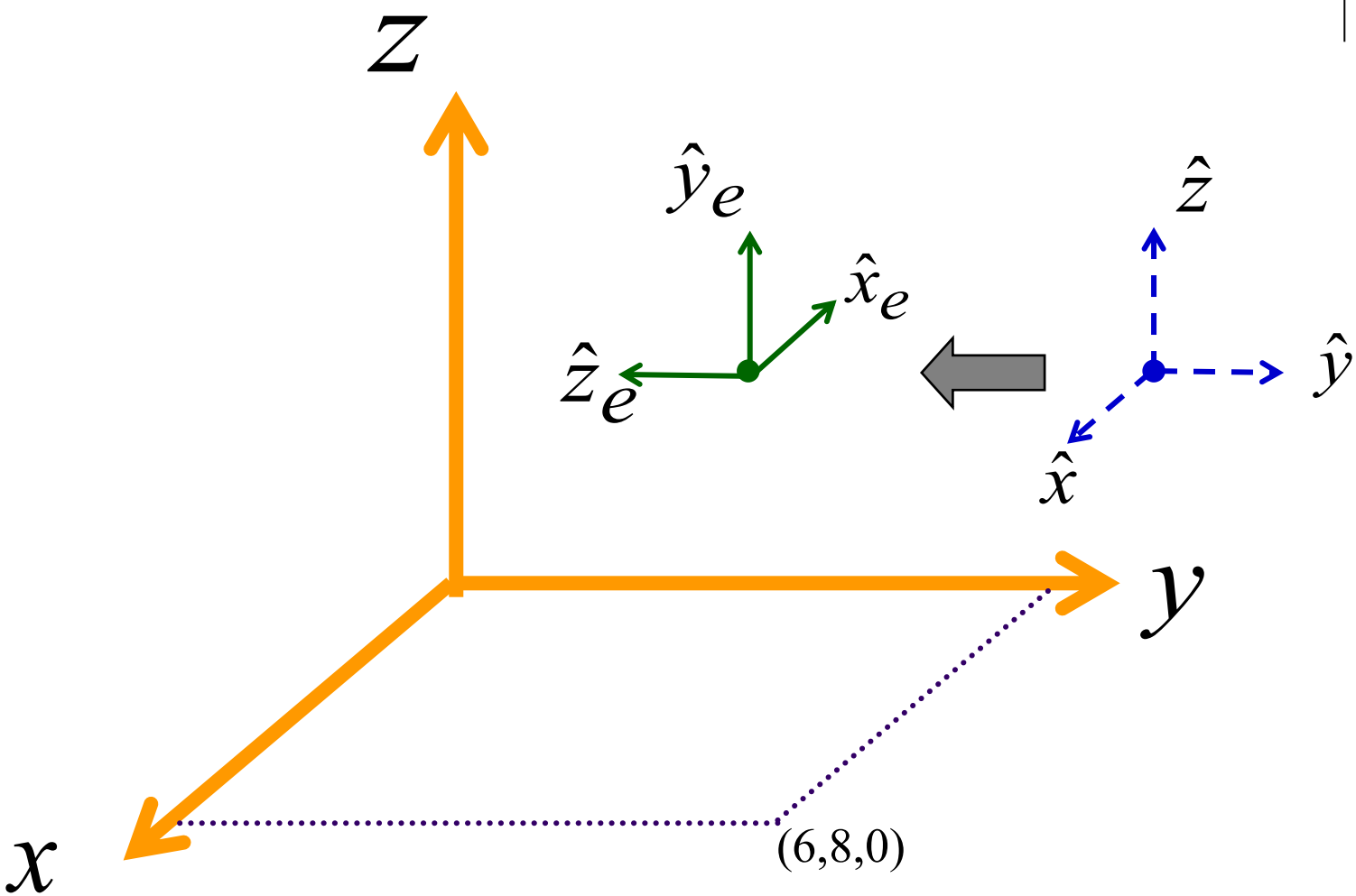
Translate Origin by T_1



Ex: Simple *Viewing Transf*

$$T_1 = \begin{bmatrix} 1 & 0 & 0 & -6 \\ 0 & 1 & 0 & -8 \\ 0 & 0 & 1 & -7.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

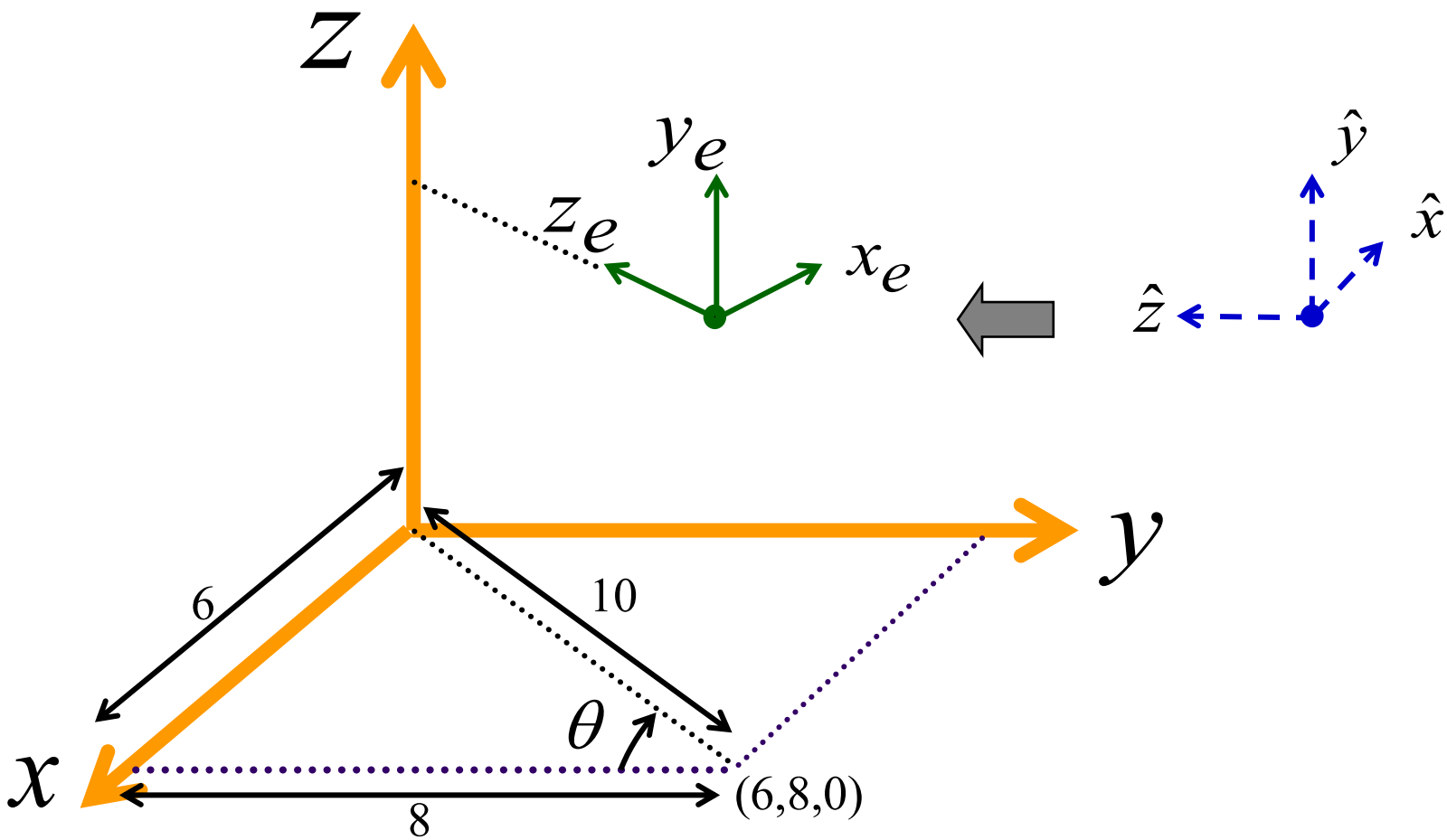
Make *LH Coord* System with T_2



Make *LH Coord System* with T_2

$$T_2 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotate about y with T_2



Ex: Simple *Viewing Transf*

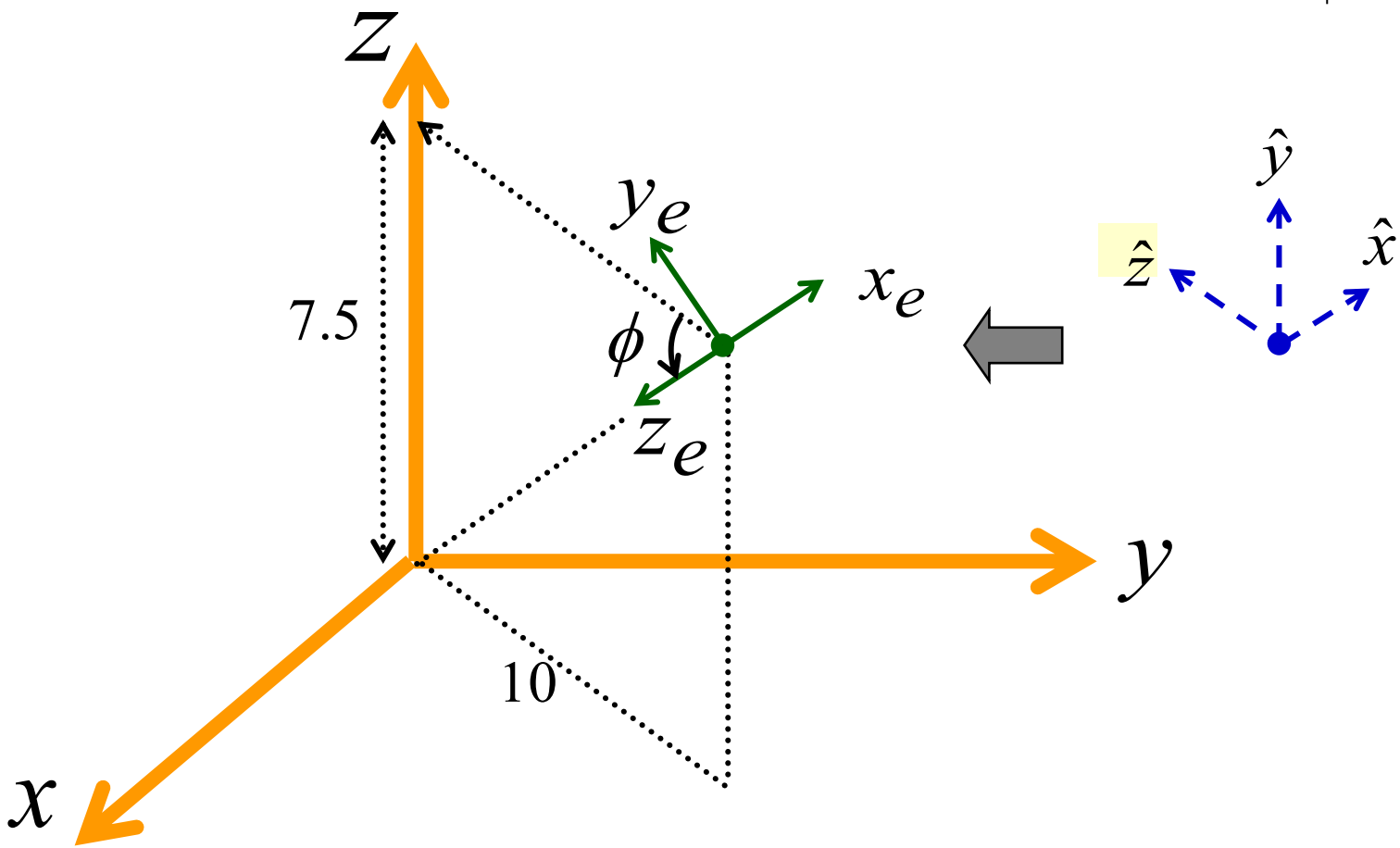
$$T_3 = \begin{bmatrix} .8 & 0 & -.6 & 0 \\ 0 & 1 & 0 & 0 \\ .6 & 0 & .8 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where,

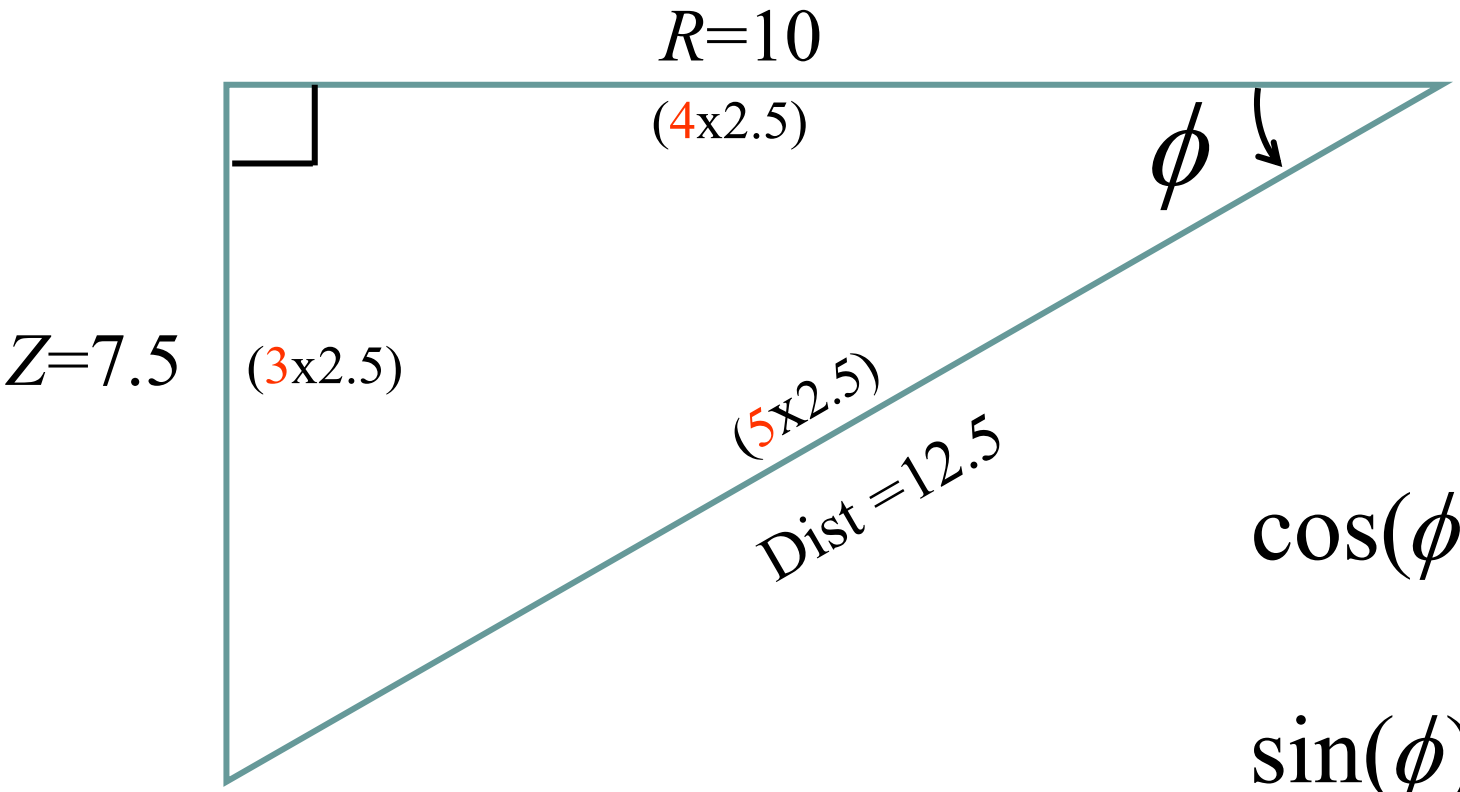
$$\cos(\theta) = 8/10$$

$$\sin(\theta) = 6/10$$

Rotate about x-axis with T_4



Look at (3-4-5) *Right Triangle*



$$\cos(\phi) = 4/5$$

$$\sin(\phi) = 3/5$$

Ex: Simple *Viewing Transf*

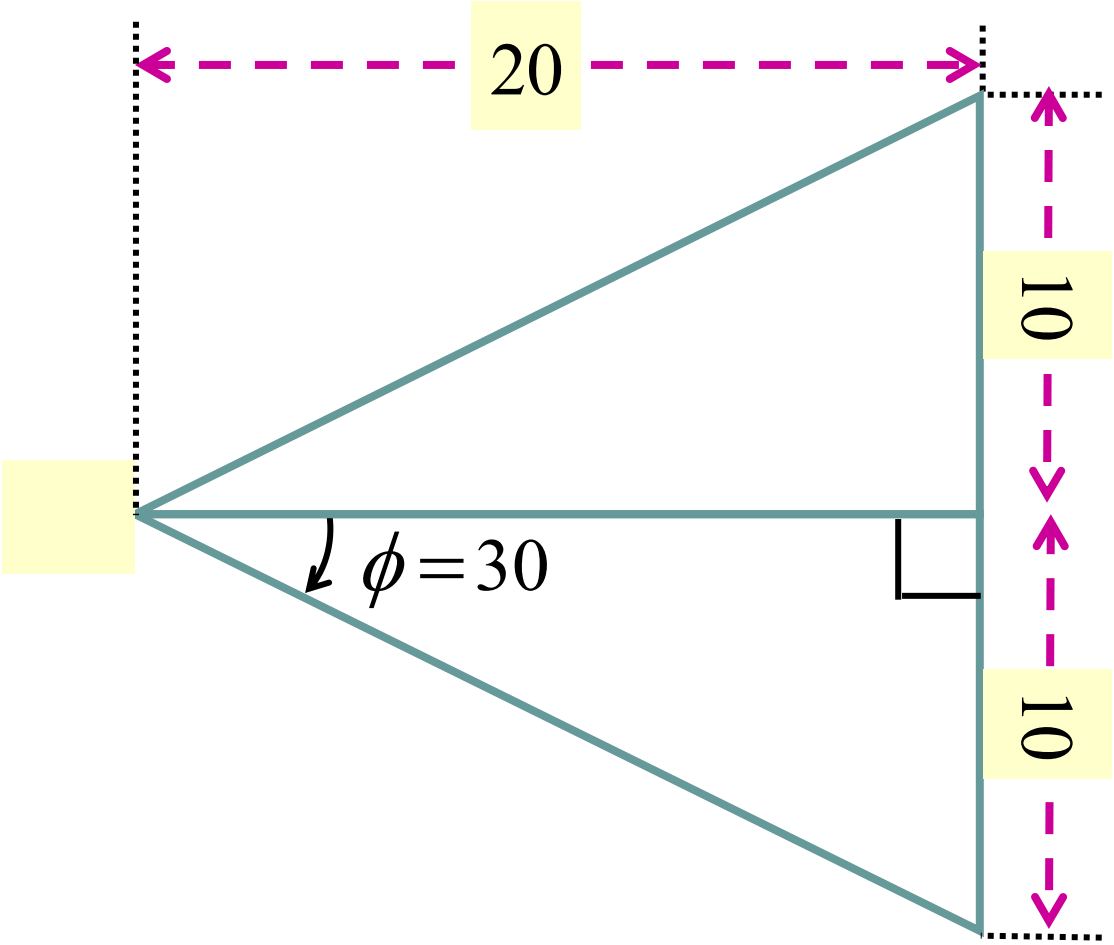
$$T_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & .8 & .6 & 0 \\ 0 & -.6 & .8 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where,

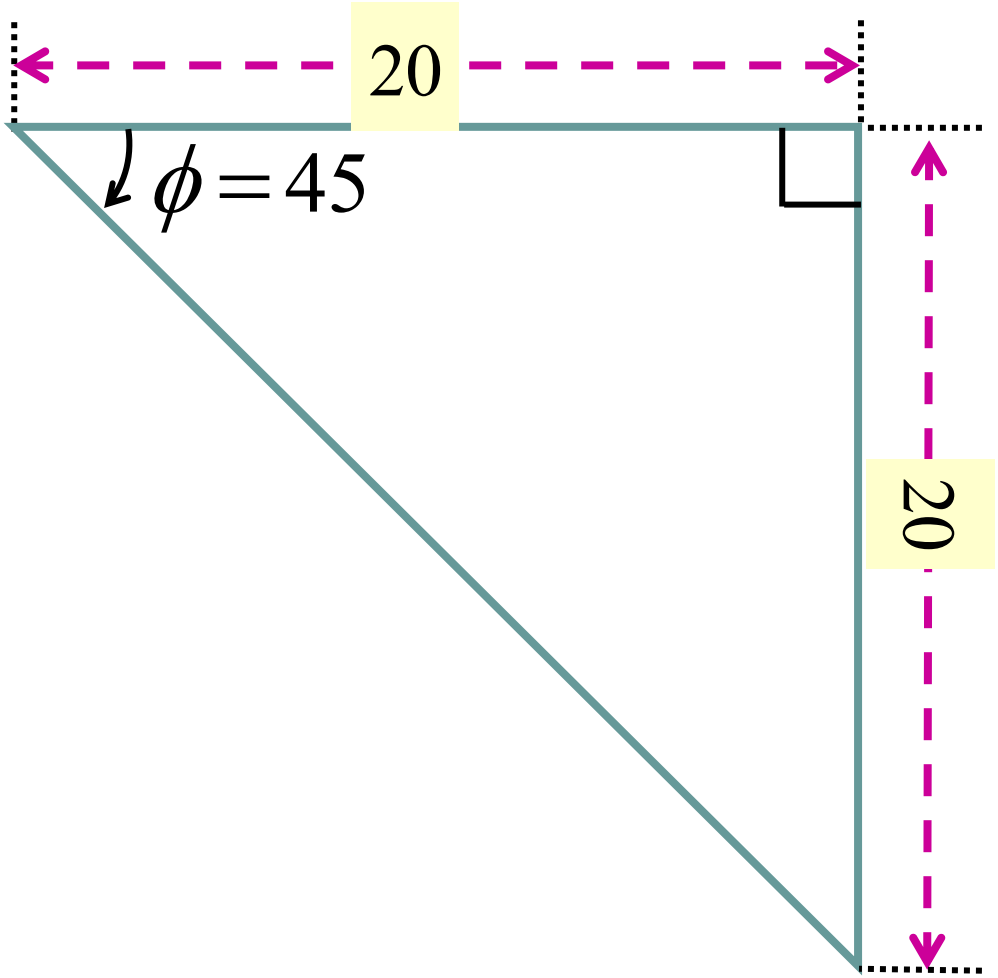
$$\cos(\phi) = 4/5$$

$$\sin(\phi) = 3/5$$

View on 20 x 20 screen, 20 away



Map to Canonical Frustum



Scale x,y by 2 *for Normalization*

Will view a 20”x 20” screen from 20” away.
Scale to standard *Viewing Frustum*.

$$N = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Simple Example: *Viewing Transf*

$$N\,T_4T_3T_2T_1 = \begin{bmatrix} -1.6 & 1.2 & 0 & 0 \\ -.72 & -.96 & 1.6 & 0 \\ -.48 & -.64 & -.6 & 12.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Clipping not needed, so *Project*

$$P_{orthographic} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Orthographic transformation zeros *z-coordinate.*

Transformation of Cube

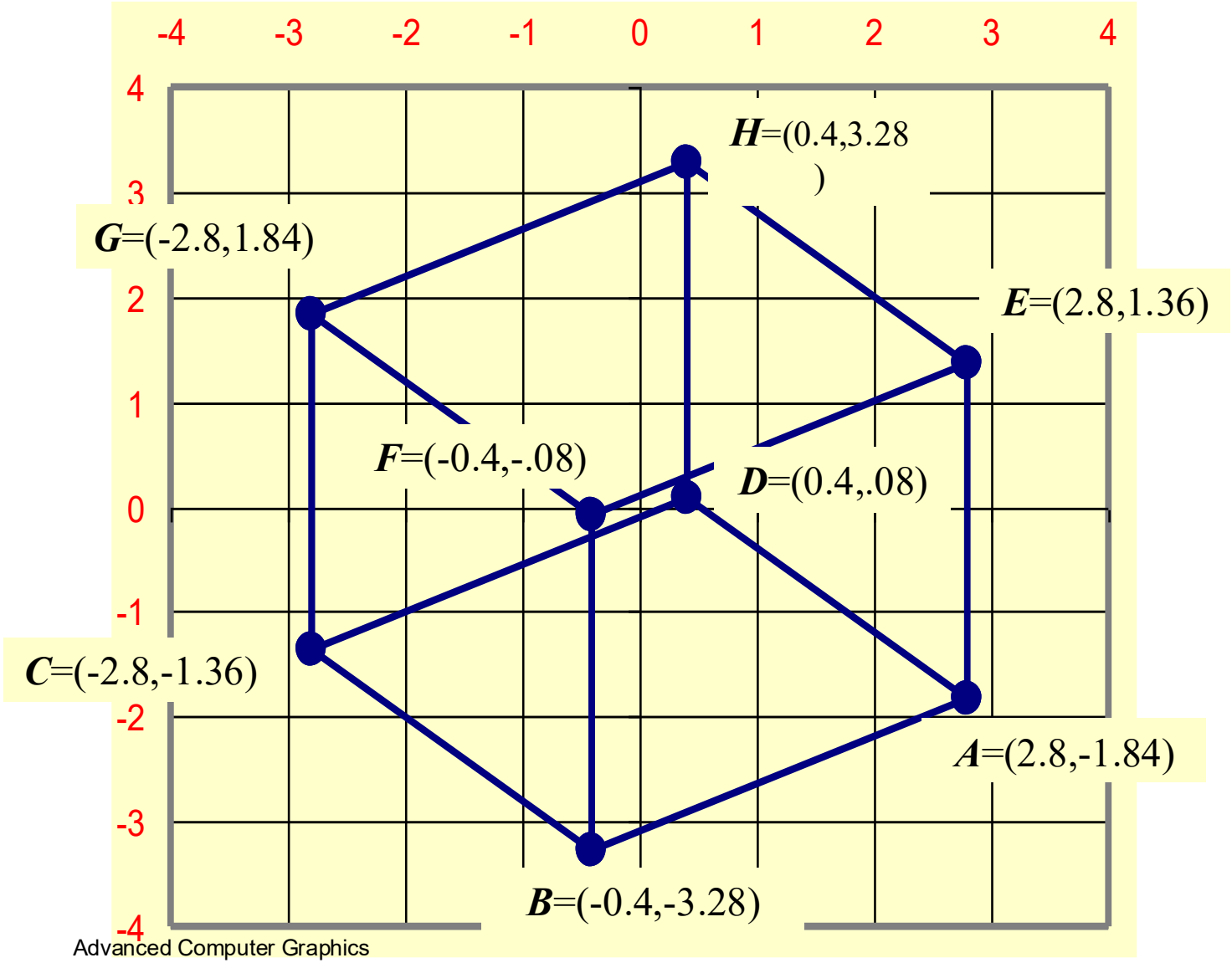
$$\begin{bmatrix} -1.6 & 1.2 & 0 & 0 \\ -.72 & -.96 & 1.6 & 0 \\ -.48 & -.64 & -.6 & 12.5 \\ 0 & 0 & 0 & +1 \end{bmatrix} \begin{bmatrix} -1 & +1 & +1 & -1 & -1 & +1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ -1 & -1 & -1 & -1 & +1 & +1 & +1 & +1 \\ +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} 2.8 & -0.4 & -2.8 & 0.4 & 2.8 & -0.4 & -2.8 & 0.4 \\ -1.84 & -3.28 & -1.36 & 0.08 & 1.36 & -0.08 & 1.84 & 3.28 \\ 12.94 & 11.98 & 13.26 & 14.22 & 11.74 & 10.78 & 12.06 & 13.02 \\ +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \end{bmatrix}$$

Cube Transformed for Viewing

Pts	*A*	*B*	*C*	*D*	*E*	*F*	*G*	*H*
X	2.8	-0.4	-2.8	0.4	2.8	-0.4	-2.8	0.4
Y	-1.84	-3.28	-1.36	.08	1.36	-.08	1.84	3.28
Z	12.94	11.98	13.26	14.22	11.74	10.78	12.06	13.02

Transformed Geometry, Same *Topology*



Pt	X	Y
A	2.8	-1.84
B	-0.4	-3.28
C	-2.8	-1.36
D	0.4	08
E	2.8	1.36
F	-0.4	-.08
G	-2.8	1.84
H	0.4	3.28

A:	B	D	E
B:	A	C	F
C:	B	D	G
D:	A	C	H
E:	A	F	H
F:	B	E	G
G:	C	F	H
H:	D	E	G

Recall mapping $[a,b] \Rightarrow [-1,1]$

- *Translate* center of *interval* to *origin*

$$x \rightarrow \left[x - \frac{a+b}{2} \right]$$

- *Normalize interval* to [-1,1]

$$\left[x - \frac{a+b}{2} \right] \rightarrow \left(\frac{1}{\left(\frac{a+b}{2} \right)} \right) \left[x - \frac{a+b}{2} \right]$$

Recall mapping $[a,b] \Rightarrow [-1,1]$

- Substitute $x = a$:

$$\frac{2}{b-a}\left[\left(\frac{2a}{2}\right)-\frac{a+b}{2}\right]=\frac{2}{b-a}\left[\frac{2a-a-b}{2}\right]$$

x

$$=\frac{2}{b-a}\left[\frac{-(b-a)}{2}\right]$$

$$=\boxed{-1}$$

Recall mapping $[a,b] \Rightarrow [-1,1]$

- Substitute $x = b$:

$$\frac{2}{b-a}\left[\left(\frac{2b}{2}\right)-\frac{a+b}{2}\right]=\frac{2}{b-a}\left[\frac{2b-a-b}{2}\right]$$

$$=\frac{2}{b-a}\left[\frac{b-a}{2}\right]=\boxed{+1}$$

x

Map to (1K x 1K) Screen

$$T = \begin{bmatrix} 1 & 0 & 0 & 511 \\ 0 & 1 & 0 & 511 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Assume screen origin (0,0) at lower left. This translates old (0,0) to center of screen (511,511).

Map to (1K x 1K) Screen

$$S_{xy} = \begin{bmatrix} 511 & 0 & 0 & 0 \\ 0 & 511 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Proper scale factor for mapping:

[-1,1] to (-511,+511)

Combine Screen Transformation

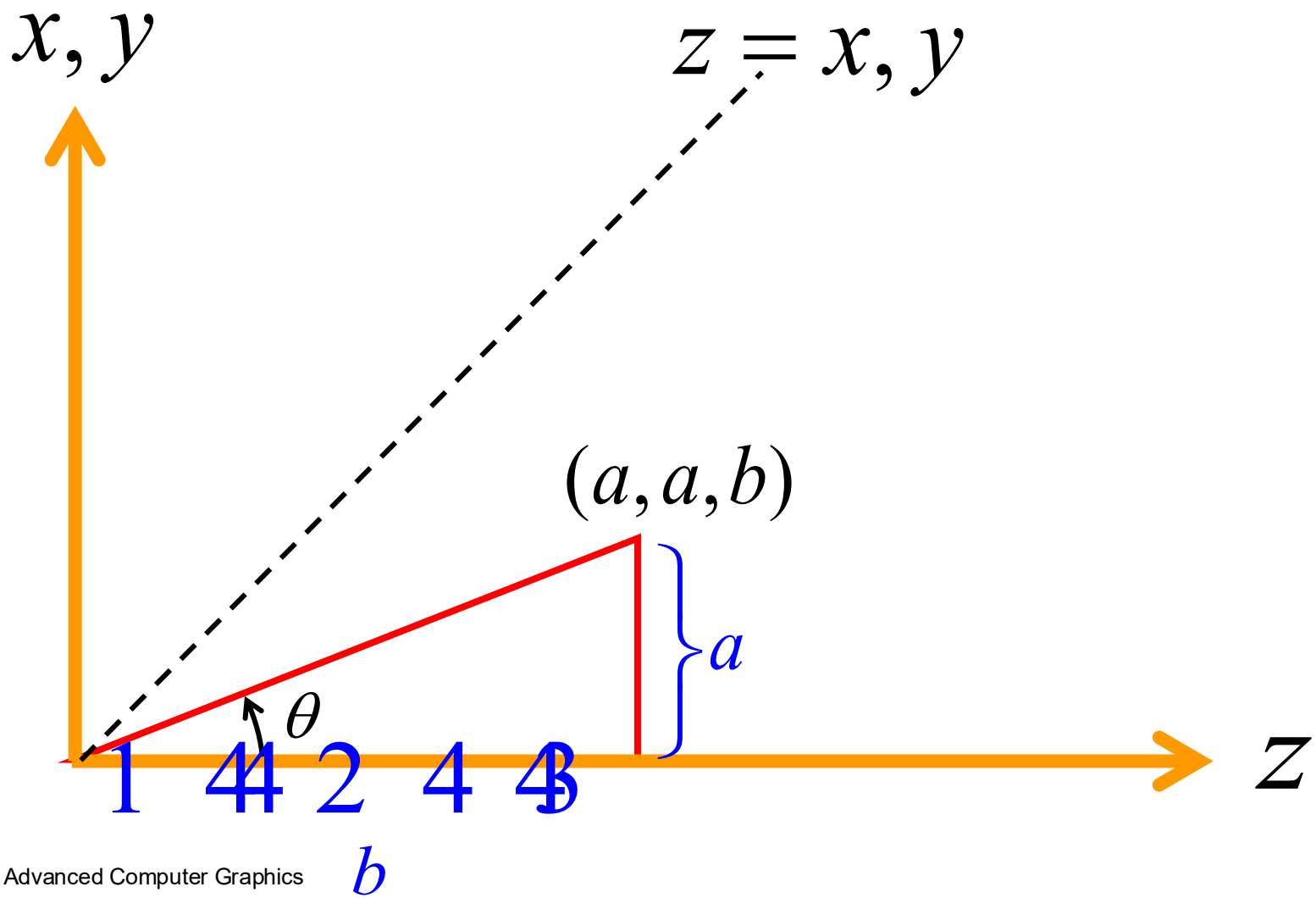
$$V = S_{xy} T_{xy} = \begin{bmatrix} 511 & 0 & 0 & 511 \\ 0 & 511 & 0 & 511 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For General Screen: $(n_x \times n_y)$ …

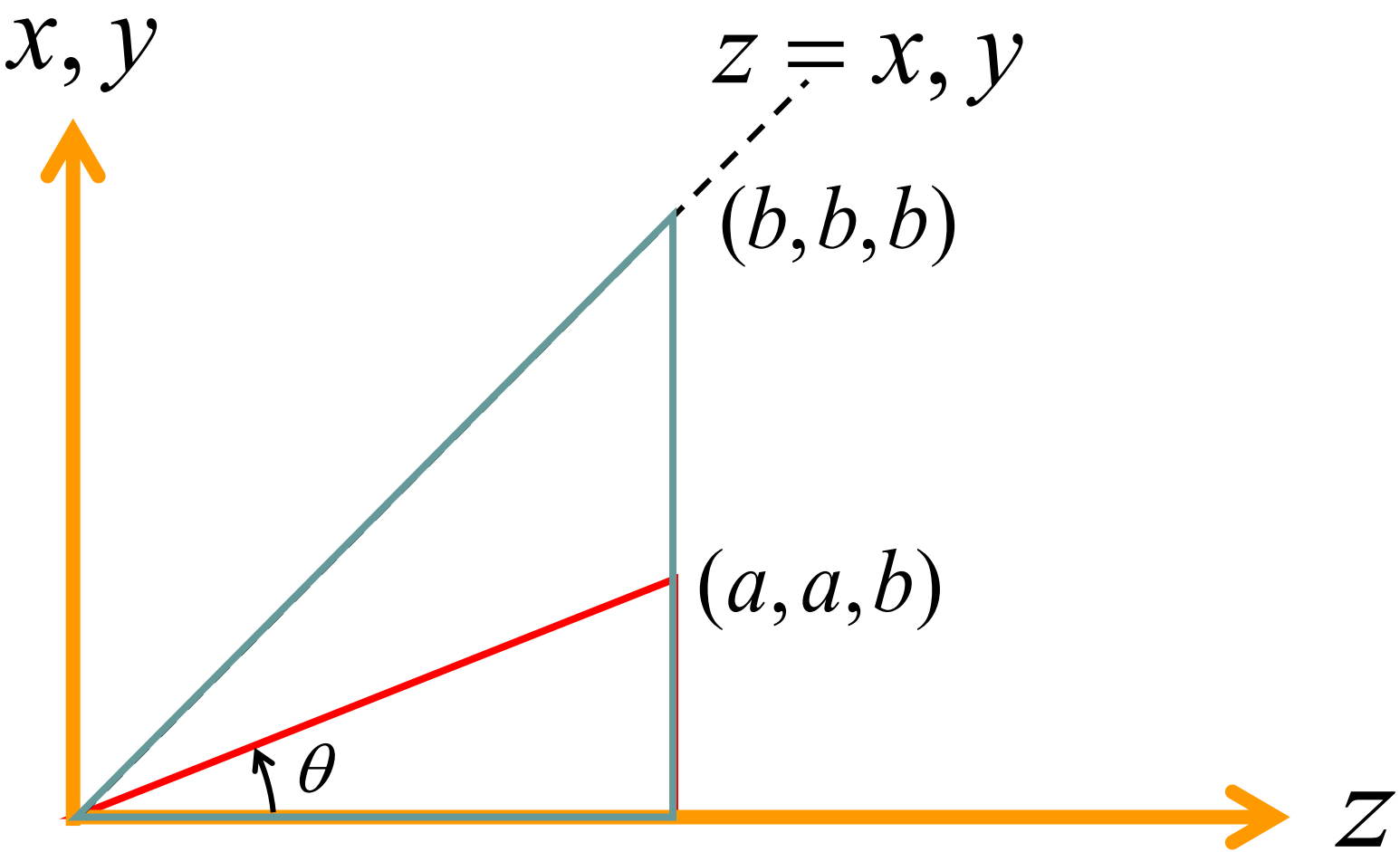
$$V = \begin{bmatrix} \left(\frac{n_x - 1}{2}\right) & 0 & 0 & \left(\frac{n_x - 1}{2}\right) \\ 0 & \left(\frac{n_y - 1}{2}\right) & 0 & \left(\frac{n_y - 1}{2}\right) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformation to Std Clipping Frustum

Transform to *Std Frustum*



Transform to *Std Frustum*



Transform to *Std Frustum*

$$\begin{bmatrix} \left(\frac{1}{\tan\theta}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{1}{\tan\theta}\right) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ a \\ b \\ 1 \end{bmatrix} = \begin{bmatrix} \cot\theta & 0 & 0 & 0 \\ 0 & \cot\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ a \\ b \\ 1 \end{bmatrix}$$

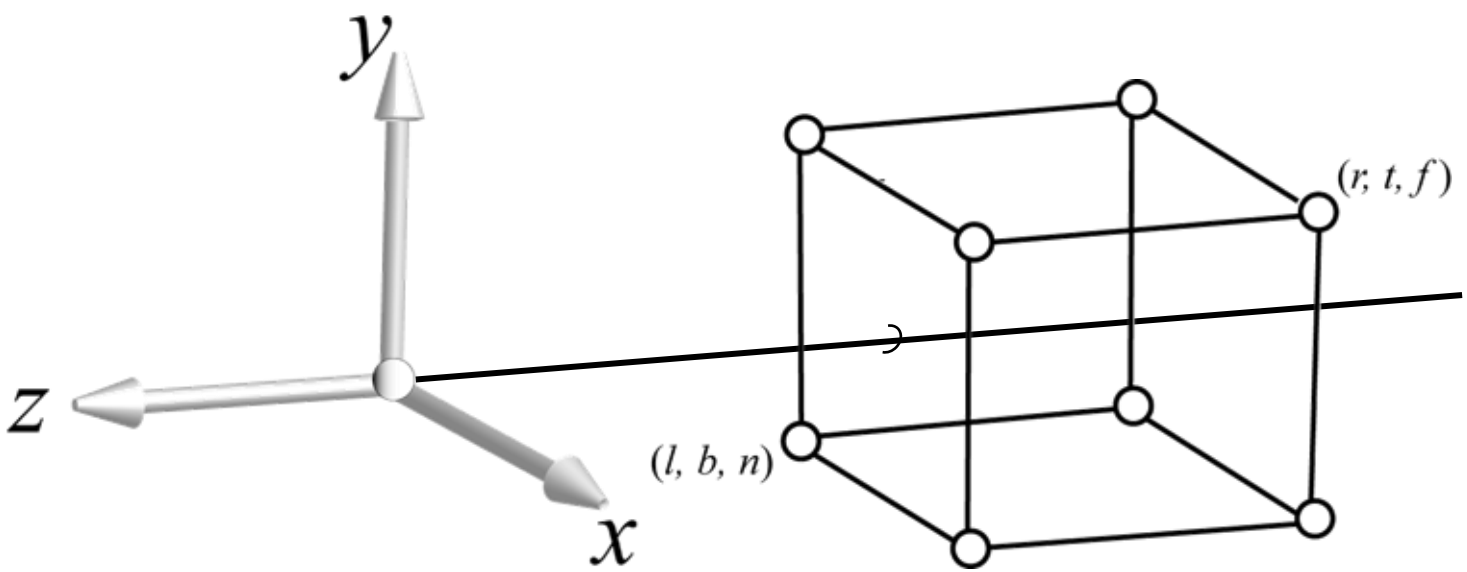
The right scale matrix to map to canonical form

Transform to *Std Frustum*

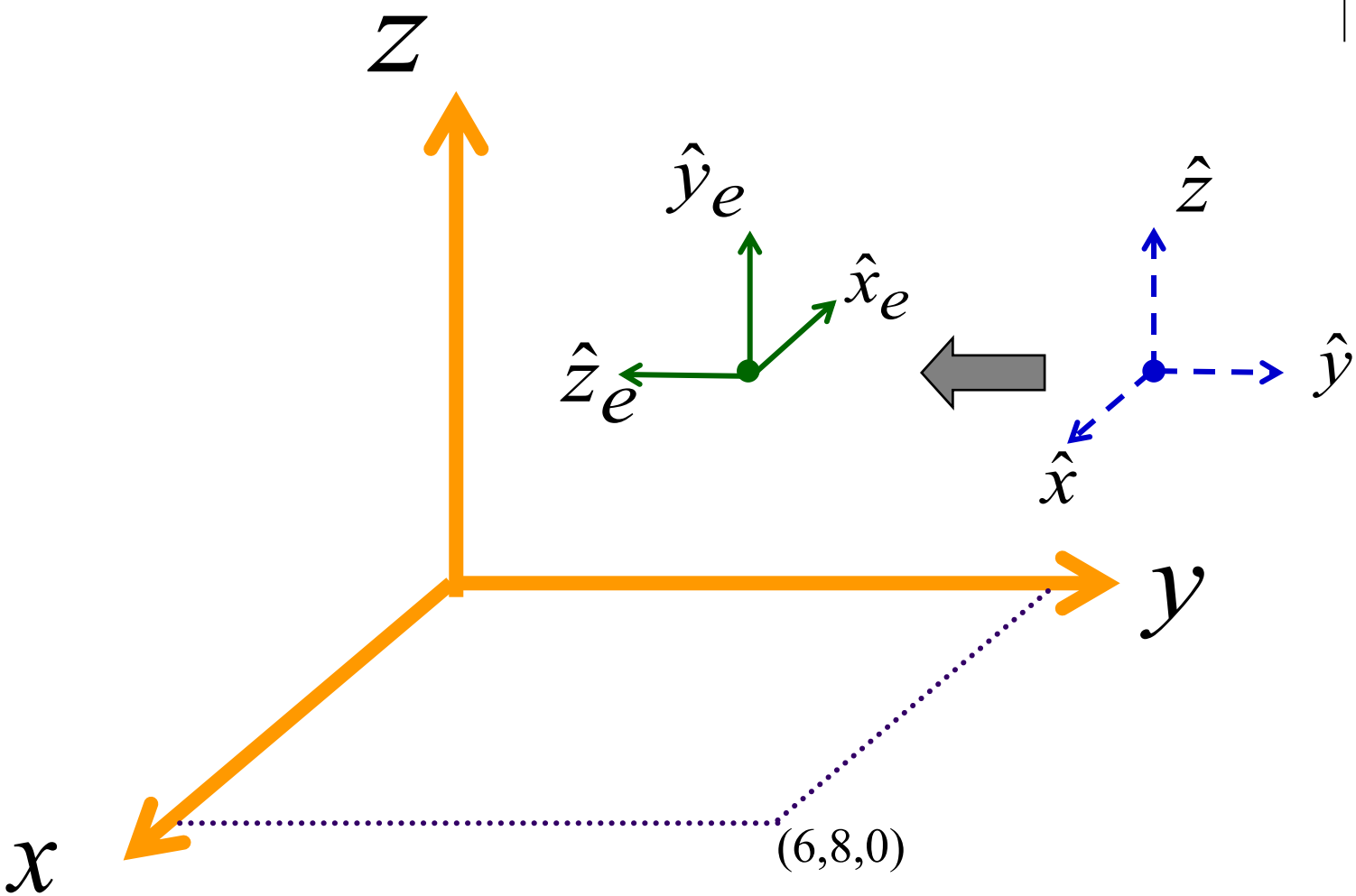
$$= \begin{bmatrix} \left(\frac{b}{a}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{b}{a}\right) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ a \\ b \\ 1 \end{bmatrix} = \begin{bmatrix} b \\ b \\ b \\ 1 \end{bmatrix}$$

Look at Transfomations

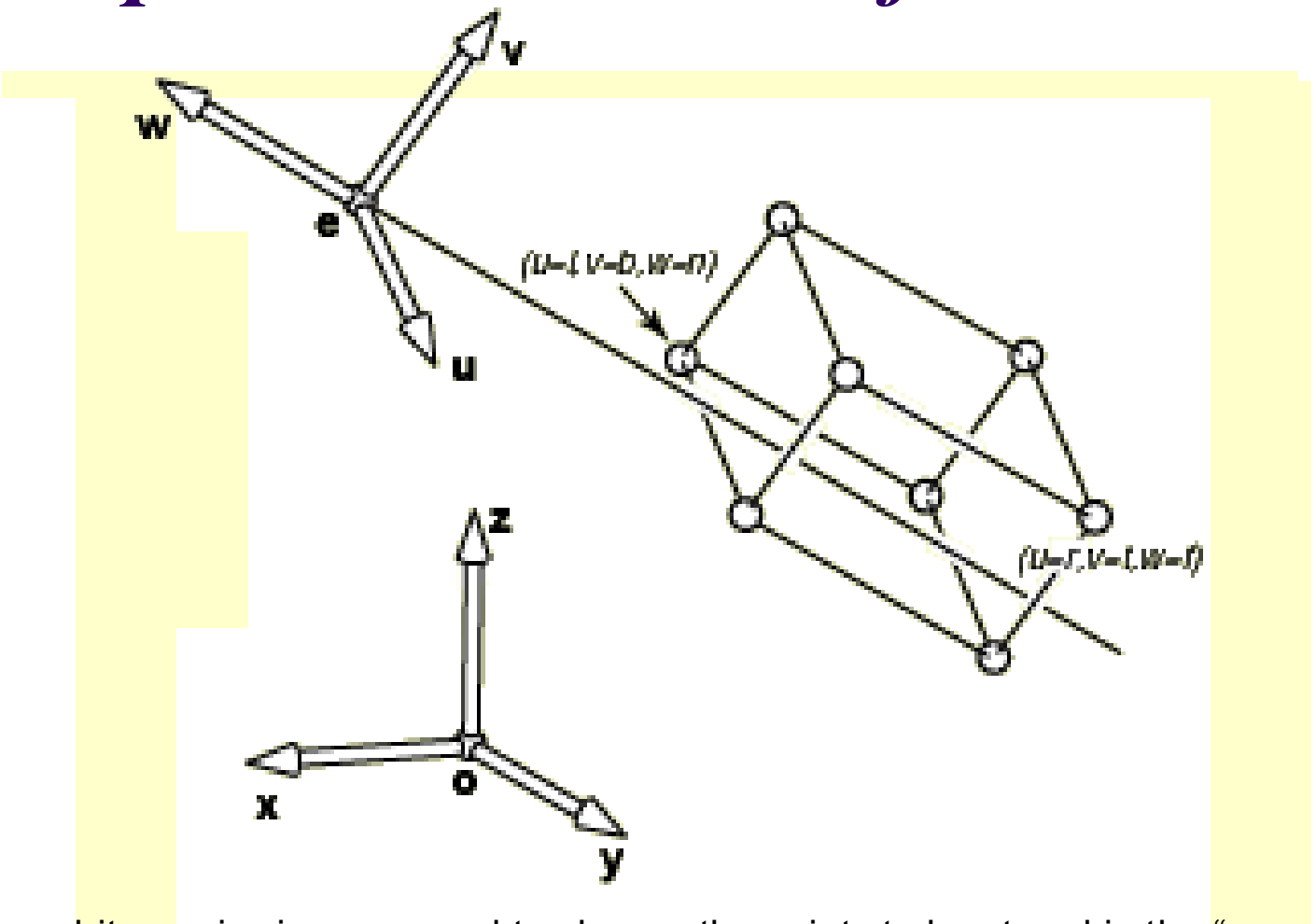
Perspective View Transformation



Make *LH Coord* System with T_2

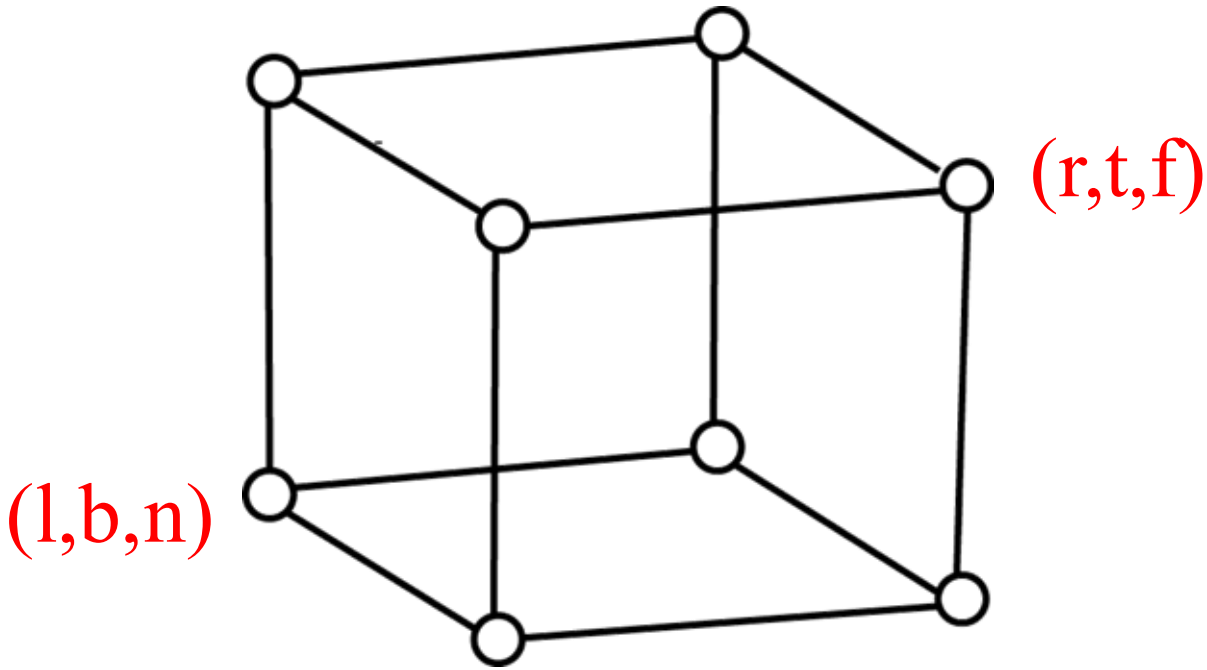


Perspective View Transformation



For arbitrary viewing, we need to change the points to be stored in the “appropriate” coordinate system. In this case it has origin **e** and offset coordinates in terms of **uvw**

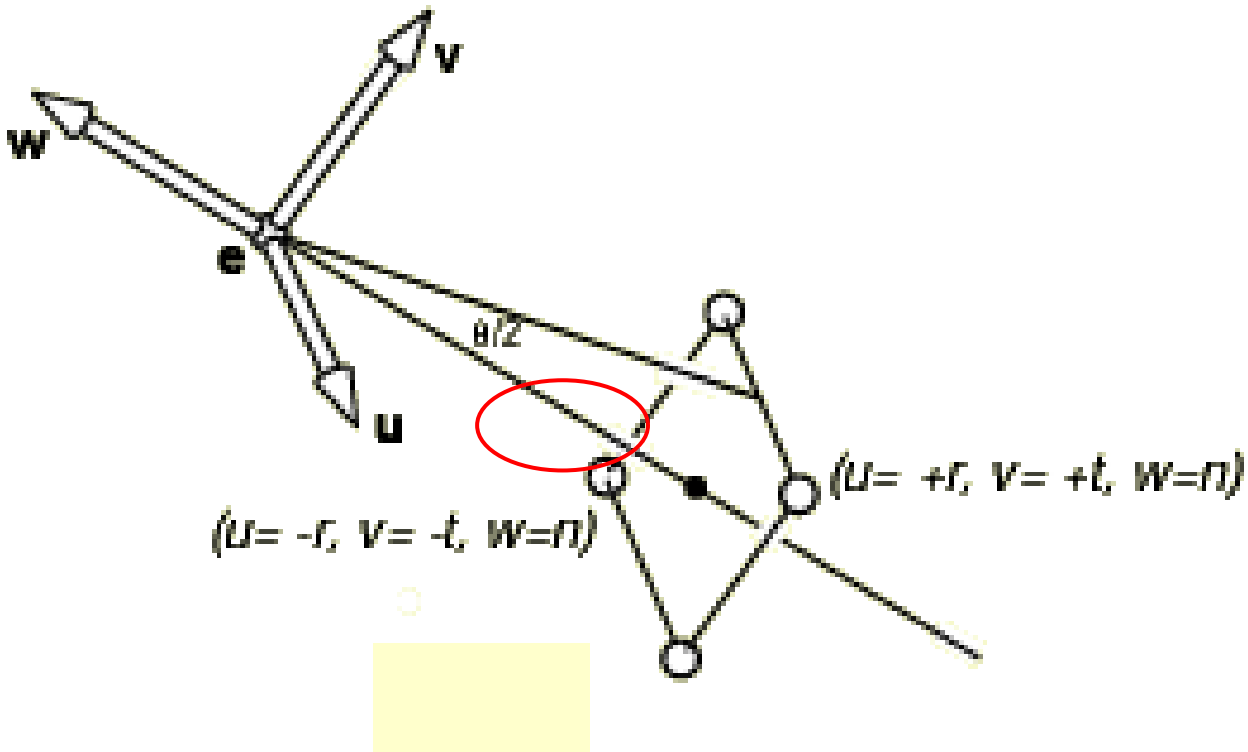
Orthographic Perspective Viewing Cube



(l,b,n) = (lower, bottom, near)

(r,t,f)= (right, top, front)

Perspective View Transformation



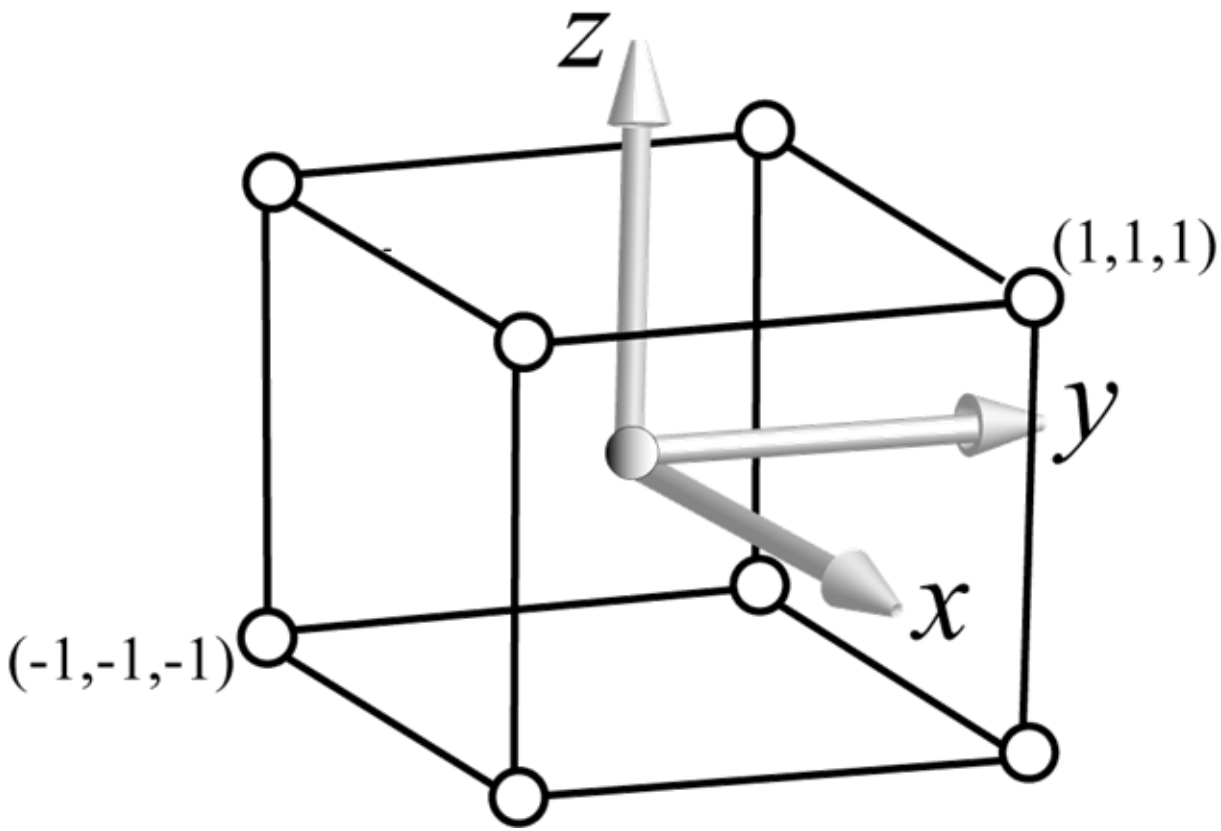
The field of view θ is the angle from the bottom of the screen to the top of the screen as measured from the eye

Factor to Make *Pyramid Standard*

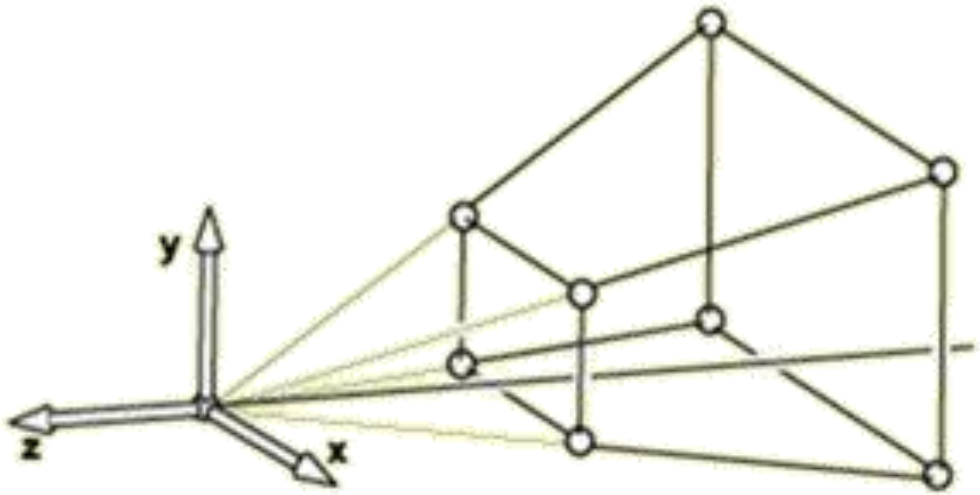
$$d = \frac{1}{\tan\left(\frac{\alpha}{2}\right)}$$

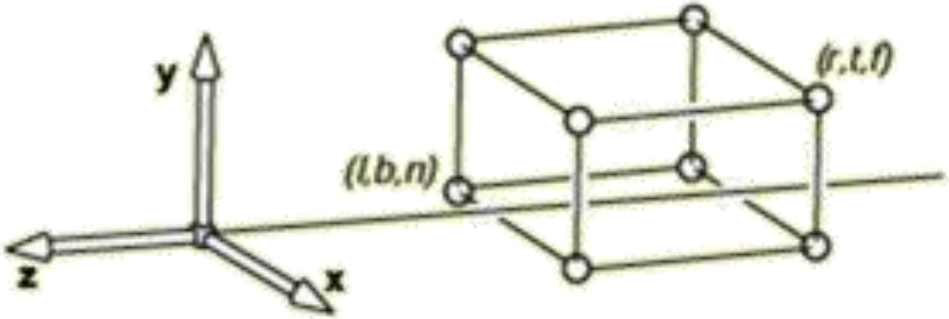
where α is the angle of the pyramid

Cube in *Model Space*



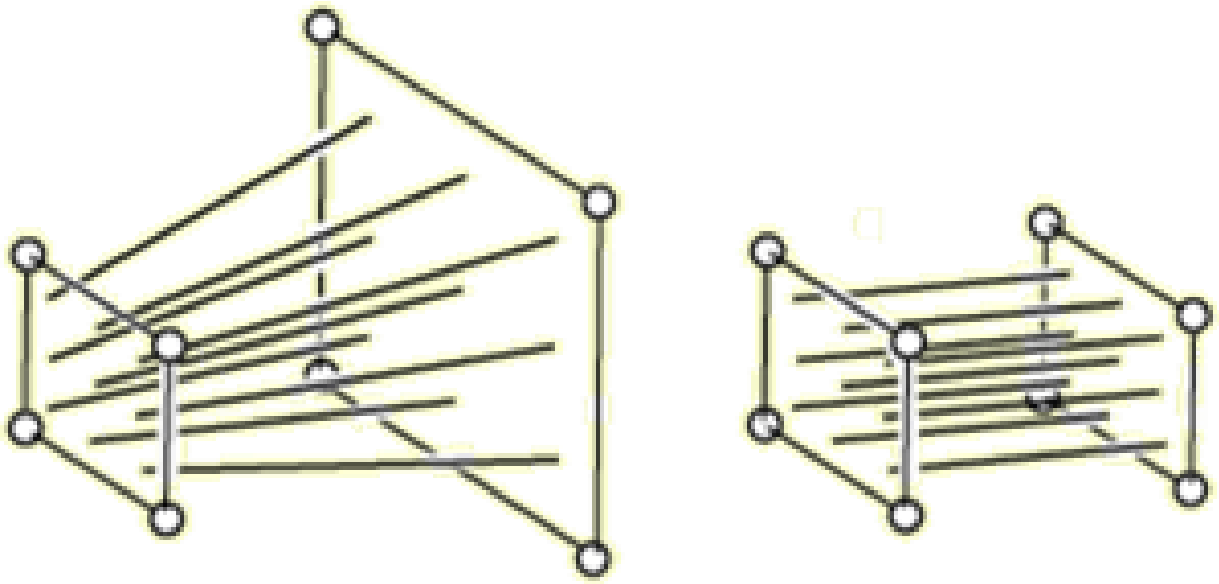
Perspective View Transformation



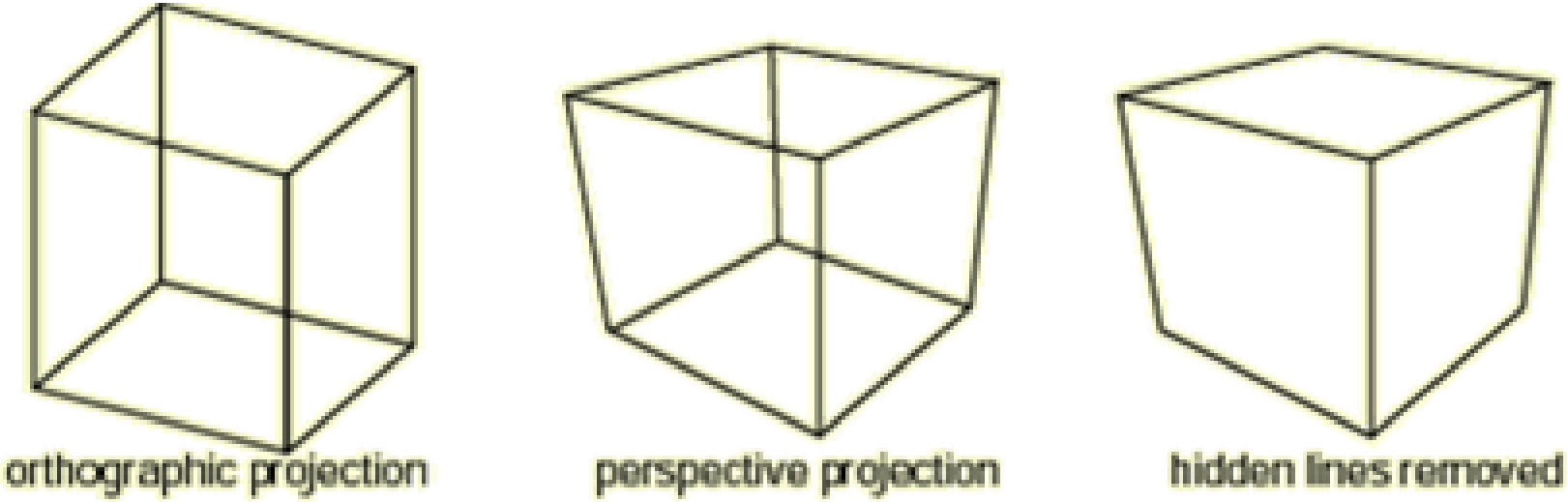


X = l: left plane
X = r: right plane
Y = b: bottom plane
Y = t: top plane
Z = n: near plane
Z = f: far plane

Perspective View Transformation

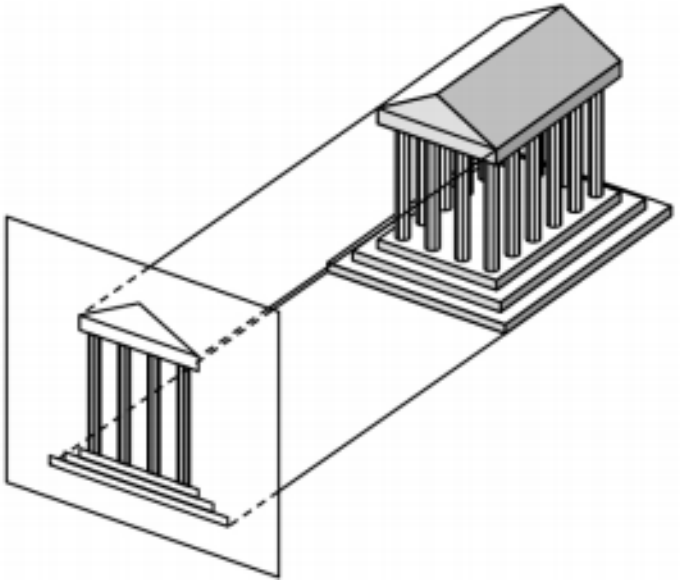


Perspective View Transformation



Orthographic Projection

- The focal point is at infinity, the rays are parallel, and orthogonal to the image plane
- When xy-plane is the image plane (x,y,z) → (x,y,0) front orthographic view

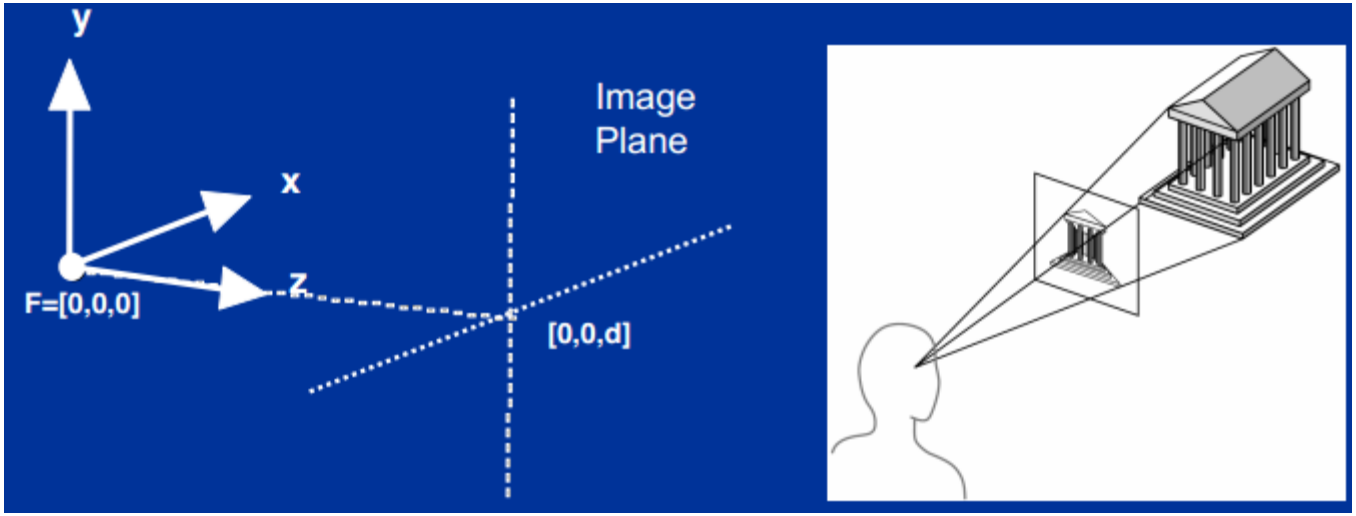


Simple Perspective Camera

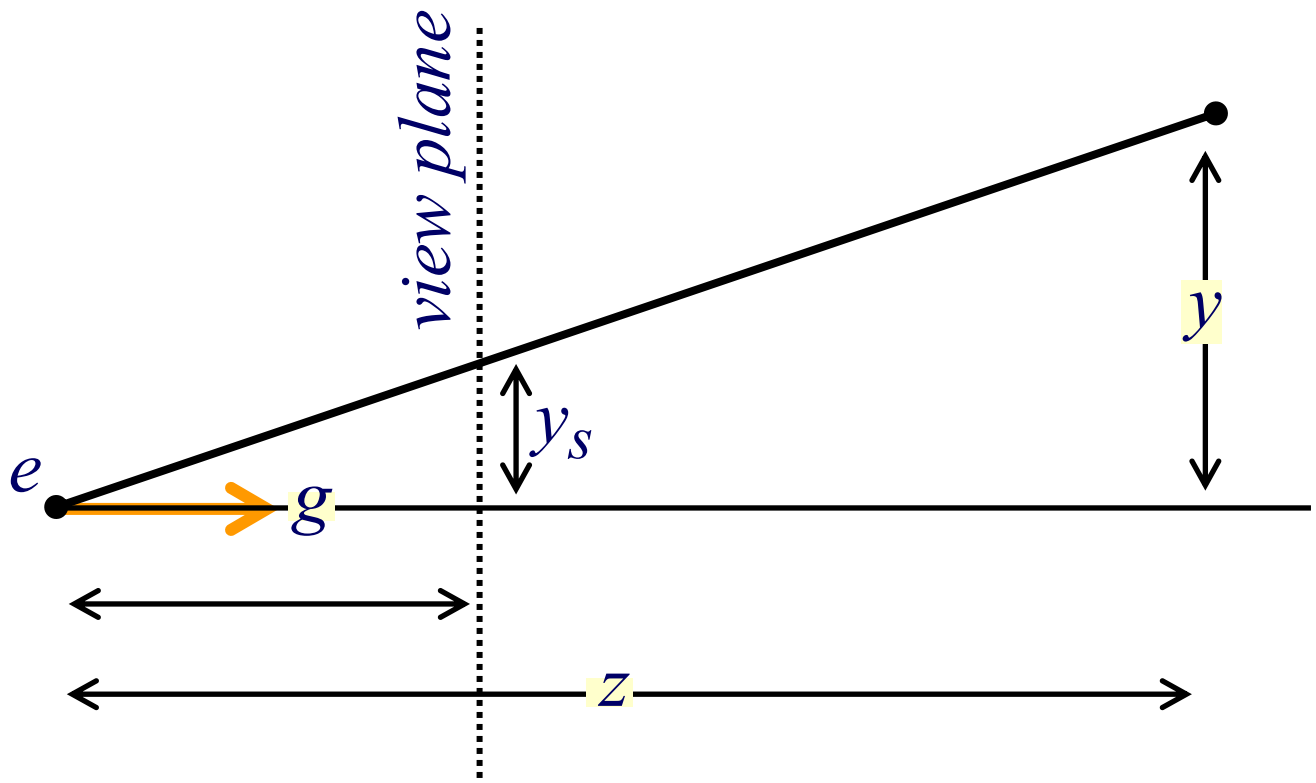
Canonical case:

- ❑ camera looks along the z-axis
- ❑ focal point is the origin
- ❑ image plane is parallel to the xy-plane at distance d

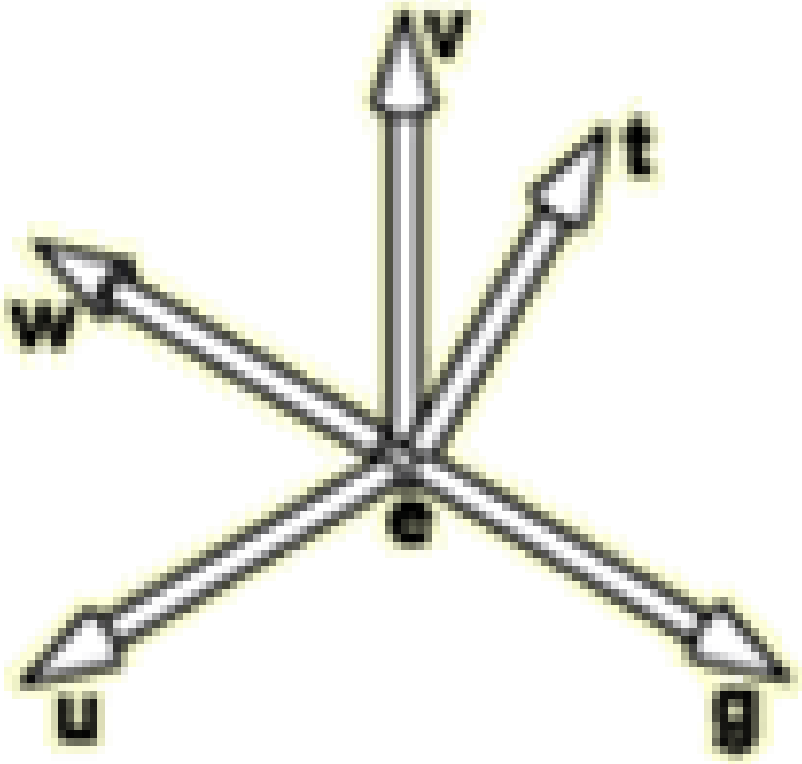
(We call d the focal length, mainly for historical reasons)



Perspective View Transformation

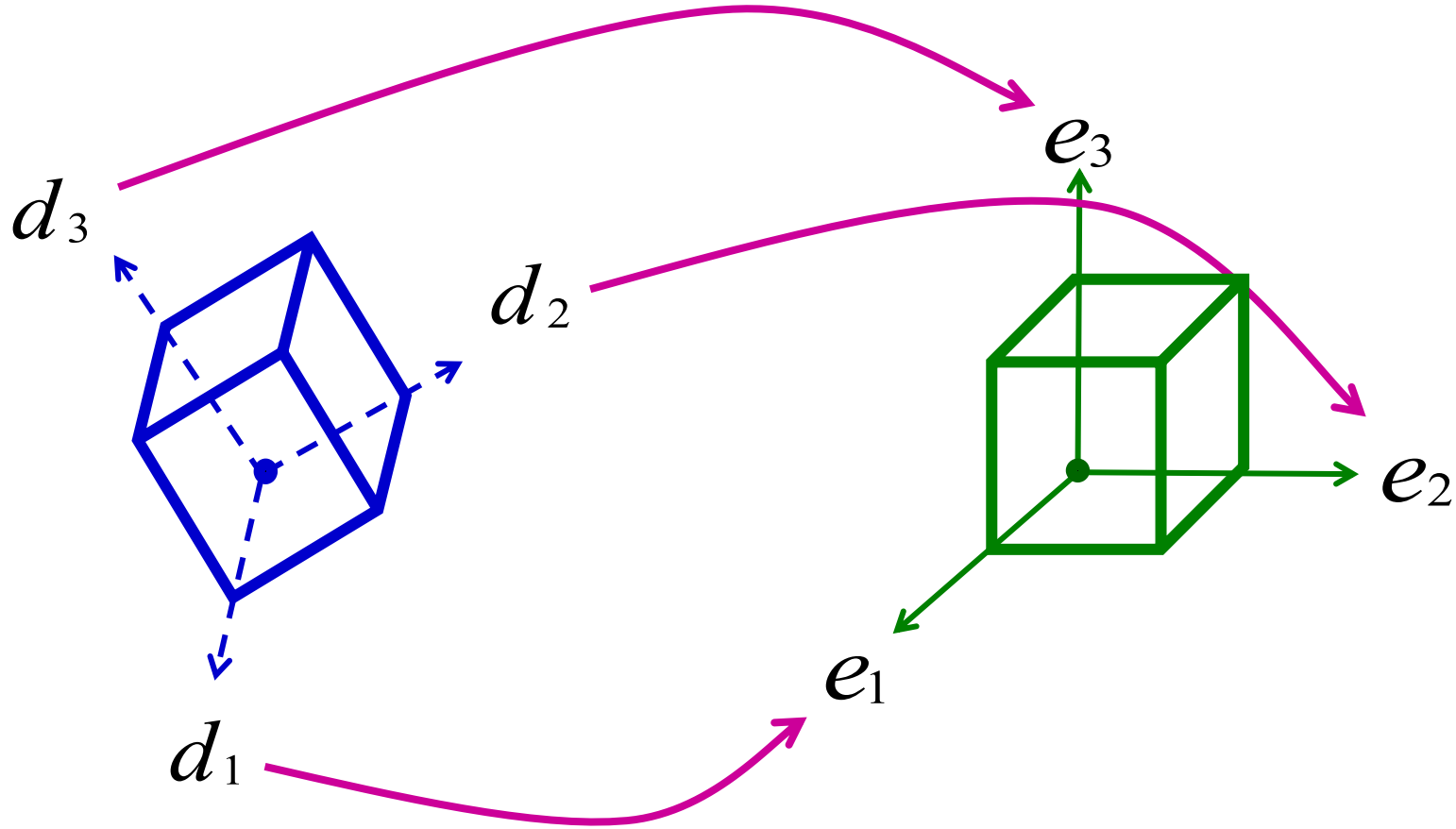


Eye Coordinate System

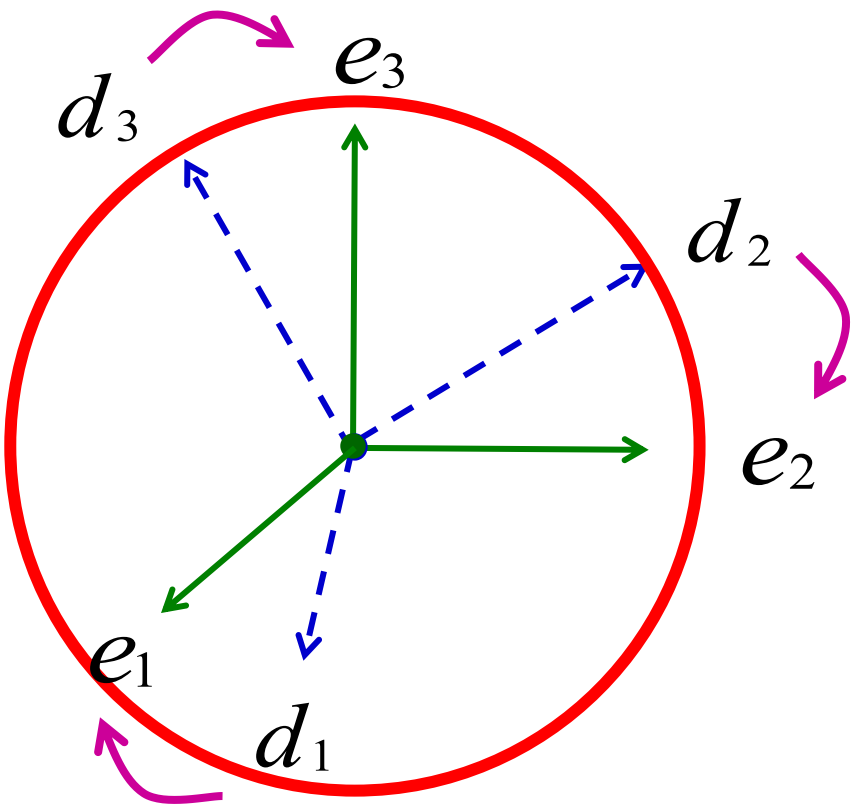


Frame rotation,

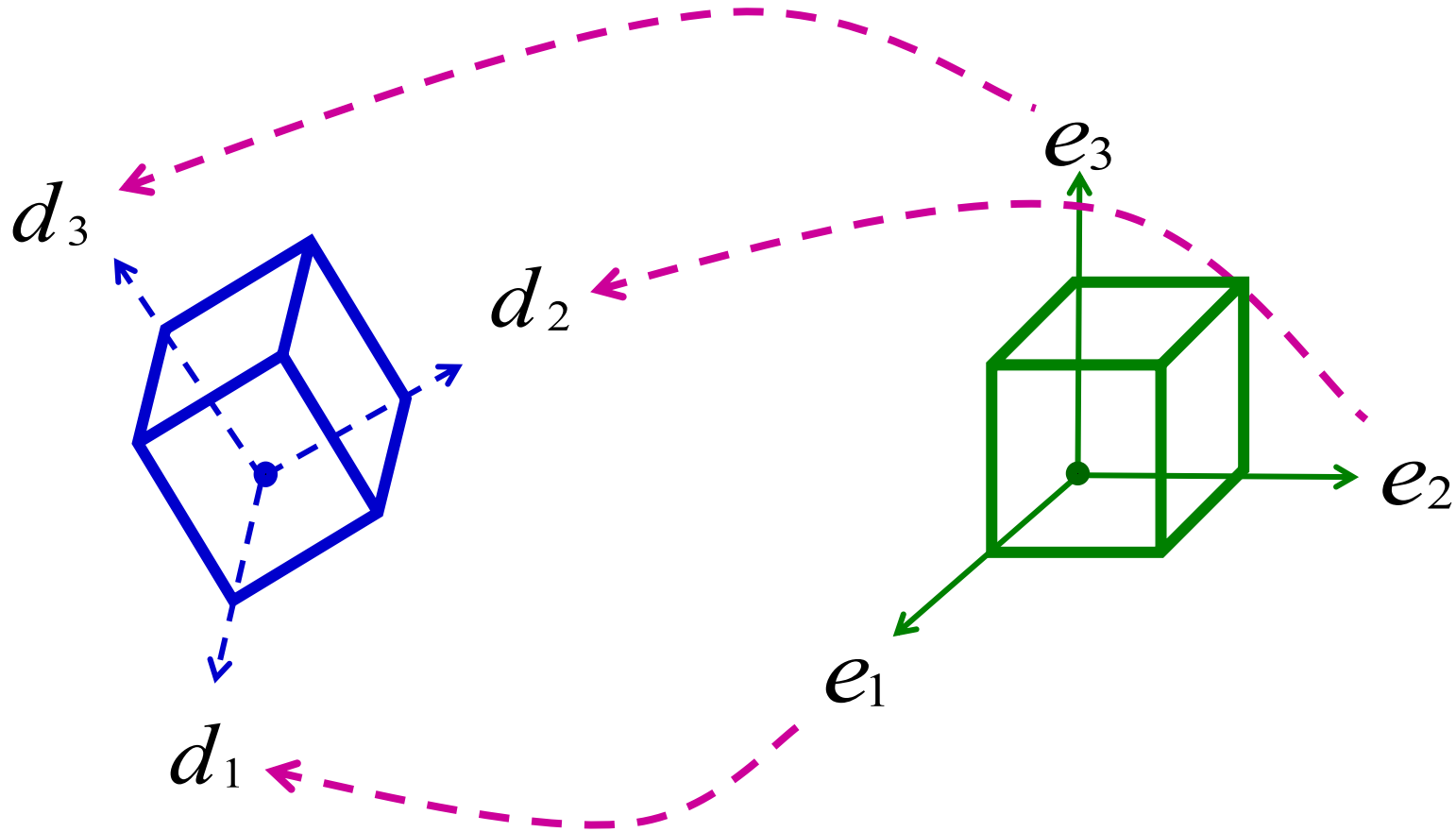
$$M: d_i \rightarrow e_i$$



Frame rotation, $M: d_i \rightarrow e_i$



Inverse Problem Easy, $M^{-1}: d_i \leftarrow e_i$



In matrix representation of M^{-1} *,*

$$\begin{bmatrix} & & \\ & M & \\ & & \end{bmatrix}^{-1} = \begin{bmatrix} d_{1x} & d_{2x} & d_{3x} \\ d_{1y} & d_{2x} & d_{3y} \\ d_{1z} & d_{2z} & d_{3z} \end{bmatrix}$$

Columns are simply images of e_i's

Rotation matrix M

- M^{-1} *columns* given by frame's *pre-image*
- *Column i* of M^{-1} is d_i

$$\begin{bmatrix} & & \\ & M & \\ & & \end{bmatrix}^{-1} = \begin{bmatrix} d_{1x} & d_{2x} & d_{3x} \\ d_{1y} & d_{2x} & d_{3y} \\ d_{1z} & d_{2z} & d_{3z} \end{bmatrix}$$

Inverse of Rotation Matrix M

- Recall, for rotation matrix R, $R^{-1} = R^t$
- So,

$$M = \left[M^{-1}\right]^{-1} = \left[M^{-1}\right]^t$$

$$\therefore \quad M = \begin{bmatrix} d_{1x} & d_{1y} & d_{1z} \\ d_{2x} & d_{2y} & d_{2z} \\ d_{3x} & d_{3y} & d_{3z} \end{bmatrix} \begin{array}{l} \leftarrow d_1 \\ \leftarrow d_2 \\ \leftarrow d_3 \end{array}$$

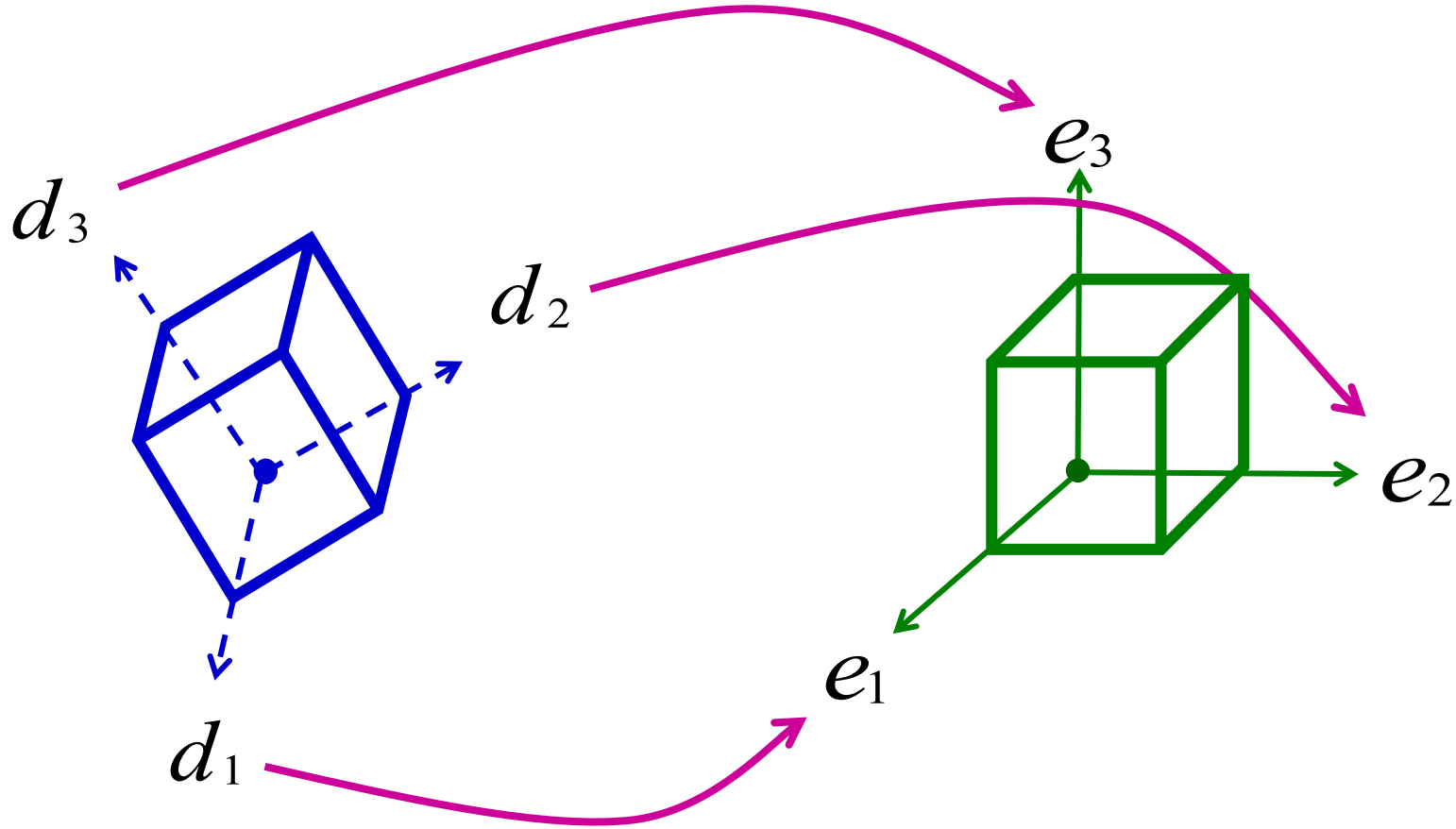
Rotation Matrix M

- *Row i* is simply d_i
- Simply write down M !

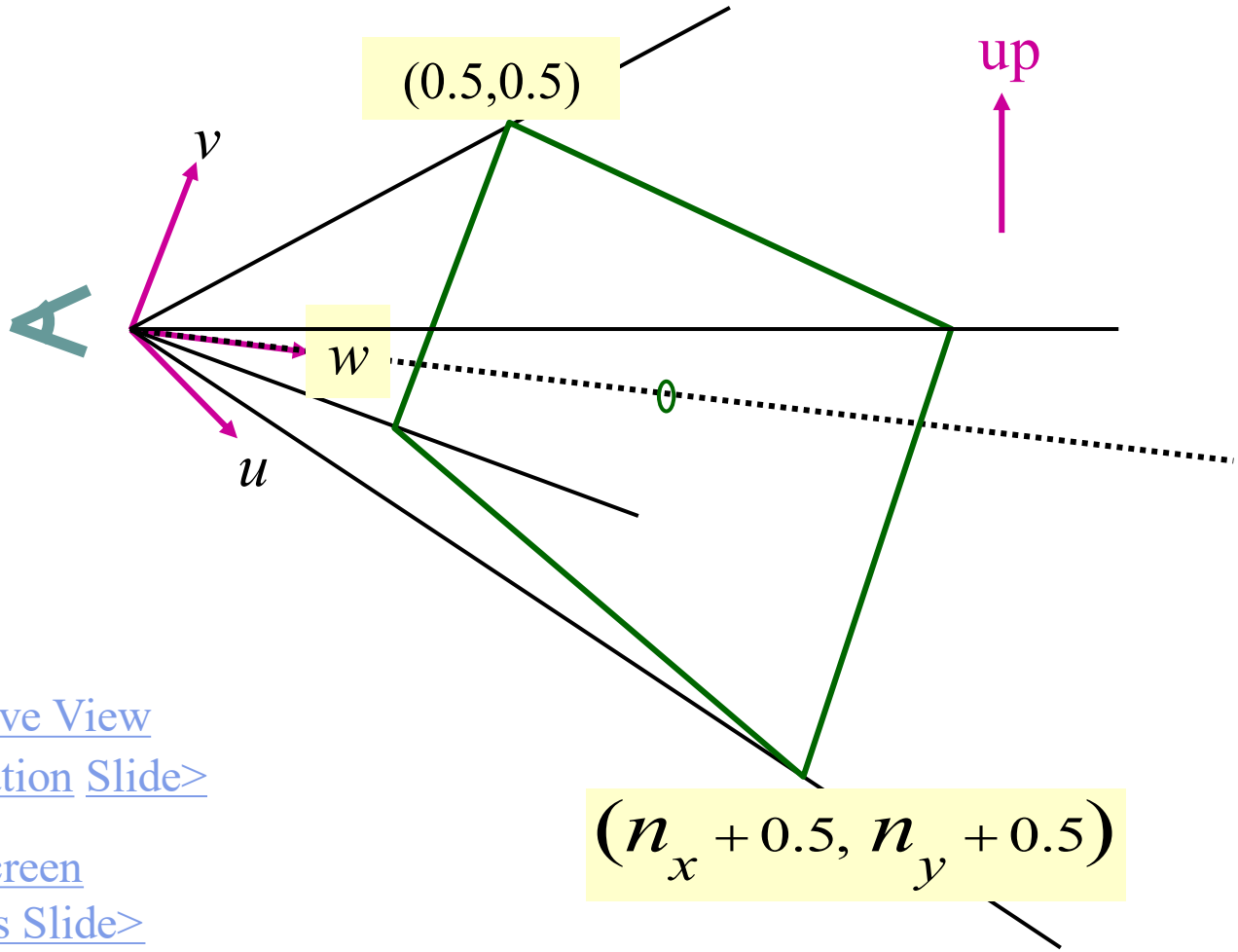
Thus,

$$M = \begin{bmatrix} d_{1_x} & d_{1_y} & d_{1_z} \\ d_{2_x} & d_{2_y} & d_{2_z} \\ d_{3_x} & d_{3_y} & d_{3_z} \end{bmatrix}$$

Frame Rotation: $d_i \rightarrow e_i$



Geometry for the *View Volume*



[<Perspective View Transformation Slide>](#)

[<Map to Screen Coordinates Slide>](#)

Revisiting Earlier Example (Contemporary Method)

Shirley's Notation

$$e = eye$$

$$t = up\ vector$$

$$at = center\ of\ \ projection$$

$$g = gaze\ direction$$

Previous Example Revisited

$$e = \begin{bmatrix} 6 & 8 & 7.5 \end{bmatrix}$$

$$t = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

$$at = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

$$g = at - e$$

$$w = -\frac{g}{\|g\|}$$

$$u = \frac{t \times w}{\|t \times w\|}$$

$$v = w \times u$$

Notation in *Shirley*

$$e = eye$$

$$t = up\ vector$$

$$at = center\ of\ \ projection$$

$$g = gaze\ direction$$

Cube Example Revisited

$$e = \begin{bmatrix} 6 & 8 & 7.5 \end{bmatrix}$$

$$t = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

$$at = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

$$g = at - e$$

$$w = -\frac{g}{\|g\|}$$

$$u = \frac{t \times w}{\|t \times w\|}$$

$$v = w \times u$$

"*Line of Sight*" *Vectors*: *g,w*

$$g = \begin{pmatrix} -6.0 & -8.0 & -7.5 \end{pmatrix}$$

$$w = -\frac{g}{\|g\|}$$

$$w = \frac{1}{12.5}\begin{pmatrix} 6.0 & 8.0 & 7.5 \end{pmatrix}$$

$$\boxed{w = \begin{pmatrix} 0.48 & 0.64 & 0.60 \end{pmatrix}}$$

Compute *Viewing Frame*: *u*

$$t \times w = \frac{1}{12.5} \det \begin{vmatrix} i & j & k \\ 0 & 0 & 1 \\ 6.0 & 8.0 & 7.5 \end{vmatrix}$$

$$= \frac{1}{12.5} \begin{pmatrix} -8.0 & 6.0 & 0.0 \end{pmatrix}$$

Compute *Viewing Frame*

$$\frac{t \times w}{\|t \times w\|} = \frac{\frac{1}{12.5}\begin{pmatrix} -8.0 & 6.0 & 0.0 \end{pmatrix}}{\frac{1}{12.5} * 10.0}$$

$$= \frac{1}{5}\begin{pmatrix} -4.0 & 3.0 & 0.0 \end{pmatrix}$$

$$\boxed{u = \begin{pmatrix} -0.8 & 0.6 & 0.0 \end{pmatrix}}$$

Compute *Viewing Frame*

$$w \times u = \left(\frac{1}{5}\right)\left(\frac{1}{12.5}\right)\det\begin{vmatrix} i & j & k \\ 6.0 & 8.0 & 7.5 \\ -4.0 & 3.0 & 0 \end{vmatrix}$$

$$= \left(\frac{1}{5}\right)\left(\frac{1}{12.5}\right)\begin{pmatrix} -22.5 & -30.0 & 50.0 \end{pmatrix}$$

$$\boxed{v = \begin{pmatrix} -0.36 & -0.48 & 0.8 \end{pmatrix}}$$

Eye Frame of Reference

$$u = \begin{bmatrix} -0.80 & 0.60 & 0.00 \end{bmatrix}$$

$$v = \begin{bmatrix} -0.36 & -0.48 & 0.80 \end{bmatrix}$$

$$w = \begin{bmatrix} 0.48 & 0.64 & 0.60 \end{bmatrix}$$

Recall, *Rotation Matrix M*

$$M = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Recall, *Rotation Matrix M*

$$M = \begin{bmatrix} -0.80 & 0.60 & 0.00 & 0 \\ -0.36 & -0.48 & 0.80 & 0 \\ 0.48 & 0.64 & 0.60 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scale to Canonical Frustum

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0.80 & 0.60 & 0.00 & 0 \\ -0.36 & -0.48 & 0.80 & 0 \\ 0.48 & 0.64 & 0.60 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} -1.60 & 1.20 & 0.00 & 0 \\ -0.72 & -0.96 & 1.60 & 0 \\ 0.48 & 0.64 & 0.60 & -12.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Same *x*-*y* values as obtained before. (z-values are difference because this is a RHS. Check!

Scale to Canonical Frustum

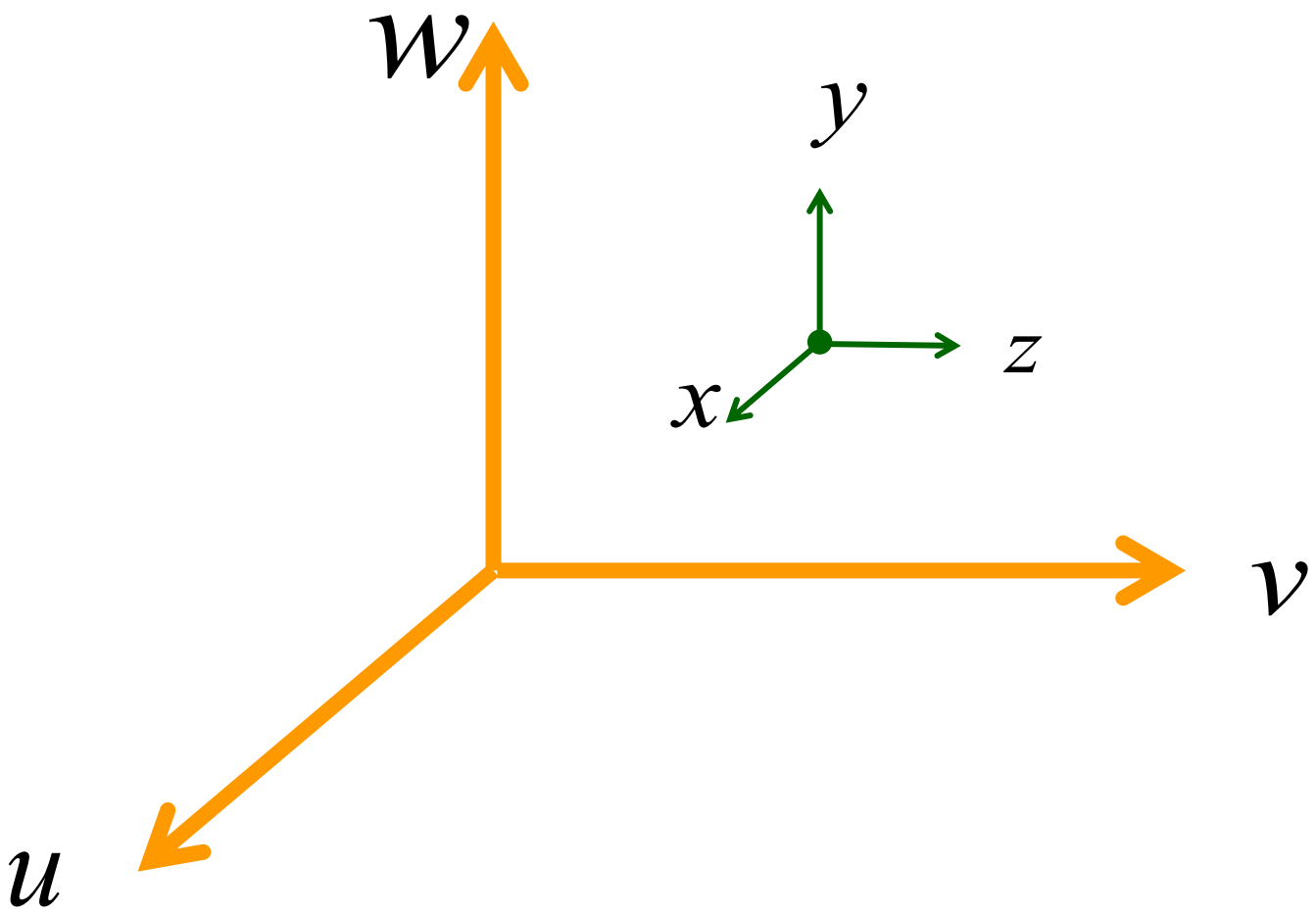
$$\begin{bmatrix} -1.60 & 1.20 & 0.00 & 0 \\ -0.72 & -0.96 & 1.60 & 0 \\ 0.48 & 0.64 & 0.60 & -12.5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} -1.6 & 1.2 & 0 & 0 \\ -.72 & -.96 & 1.6 & 0 \\ -.48 & -.64 & -.6 & 12.5 \\ 0 & 0 & 0 & +1 \end{bmatrix}$$

Same *x*-*y* values as obtained before. (z-values are difference because this is a RHS. Check!

A "True Perspective" Projection

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -12.5 \\ 0 & 0 & 0.080 & 0 \end{bmatrix}$$

View Coordinate Transformation



View Coordinate Transformation

Transforms to the view coordinate system by:

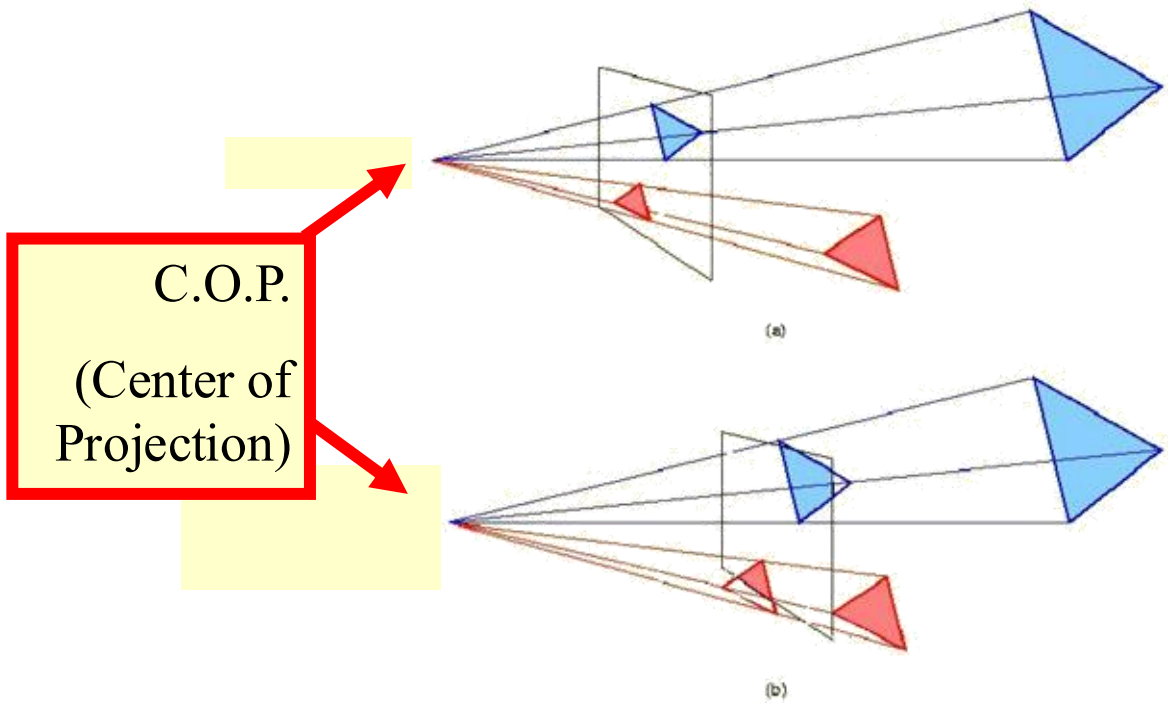
- First translating to the *eye* point.
- Then it rotates so that <*u,v,w*> become the new <*x,y,z*>, resp. The result is a *right*-handed coordinate system in Shirley

View Coordinate Transformation

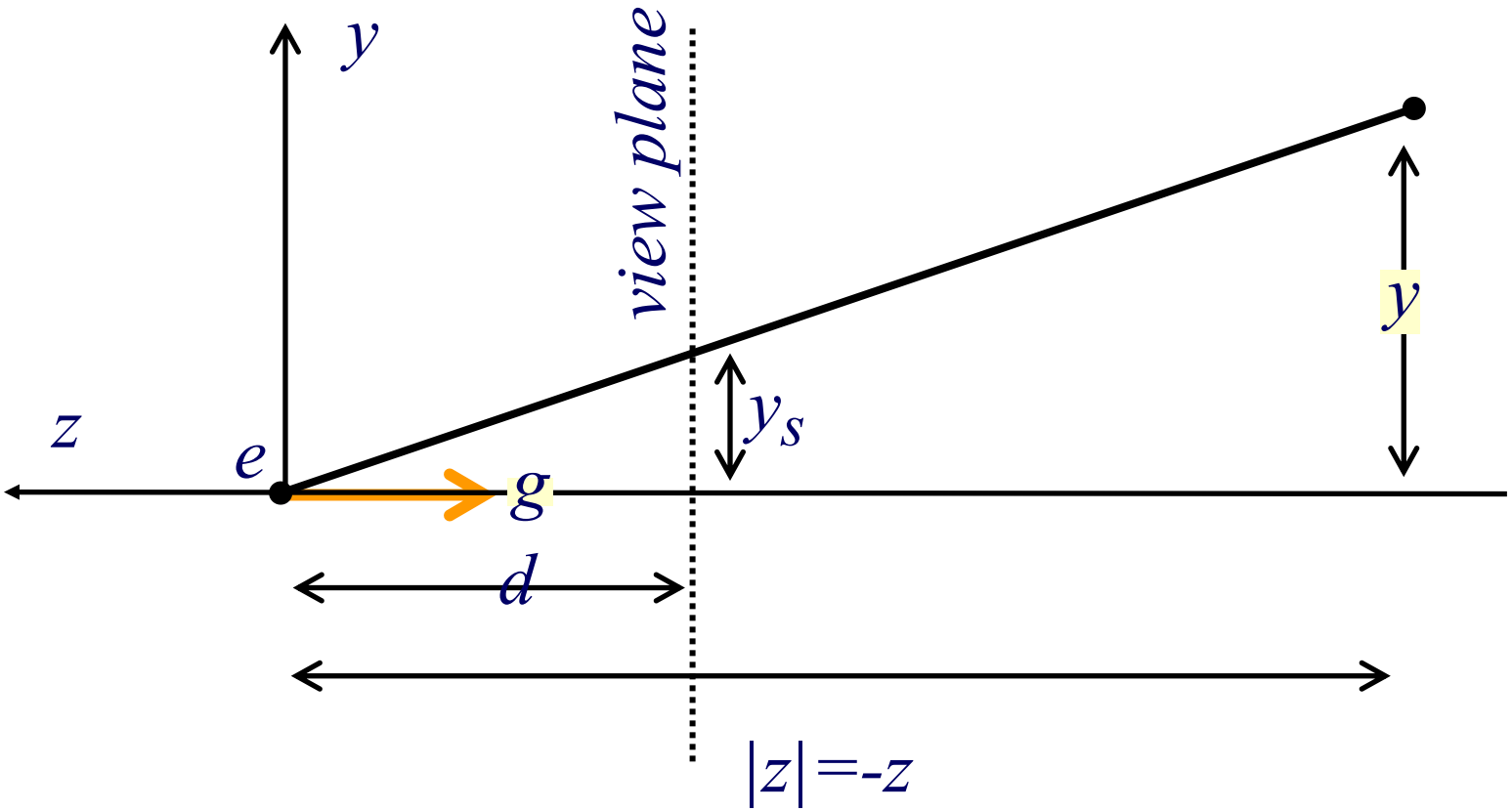
$$M_v = \begin{bmatrix} x_u & x_v & x_w & 0 \\ y_u & y_v & y_w & 0 \\ z_u & z_v & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -e_u \\ 0 & 1 & 0 & -e_v \\ 0 & 0 & 1 & -e_w \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

rotation *translation*

Projective Transformations (Perspective Projection)



Perspective View Transformation After Translating Eye to Origin



Translate Eye to Origin and Then Perform Perspective Transformation

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -d \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -d \\ 0 & 0 & -1/d & 1 \end{bmatrix}$$

perspective *translation*

d is a distance (d > 0)

Map Image Plane to Screen

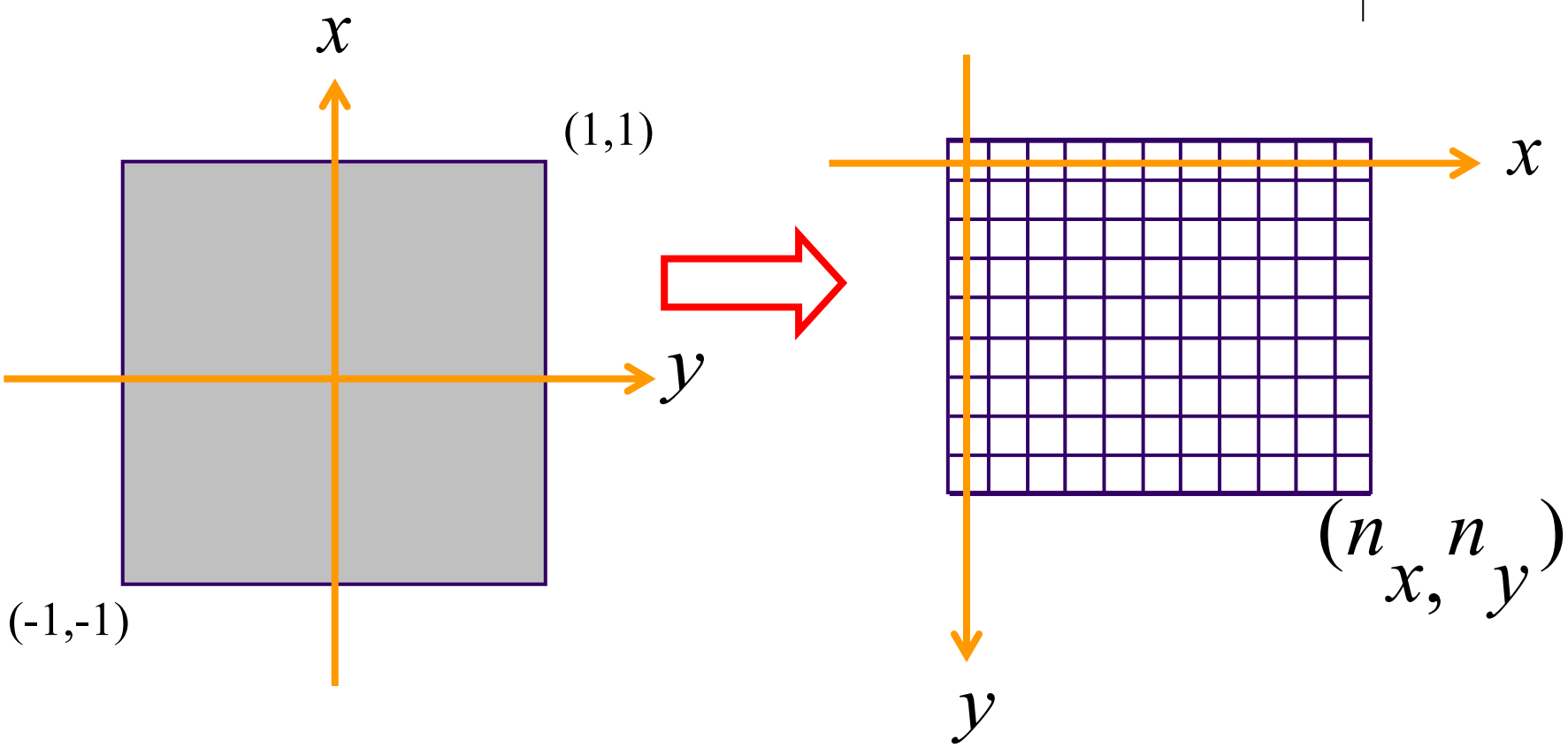


Image Plane **Screen**

Map to Screen Coordinates (For Displaying on the Screen)

$$T_2 = \begin{bmatrix} 1 & 0 & 0 & \frac{1}{2}(n_x - 1) \\ 0 & 1 & 0 & \frac{1}{2}(n_y - 1) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{2}n_x & 0 & 0 & 0 \\ 0 & -\frac{1}{2}n_y & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

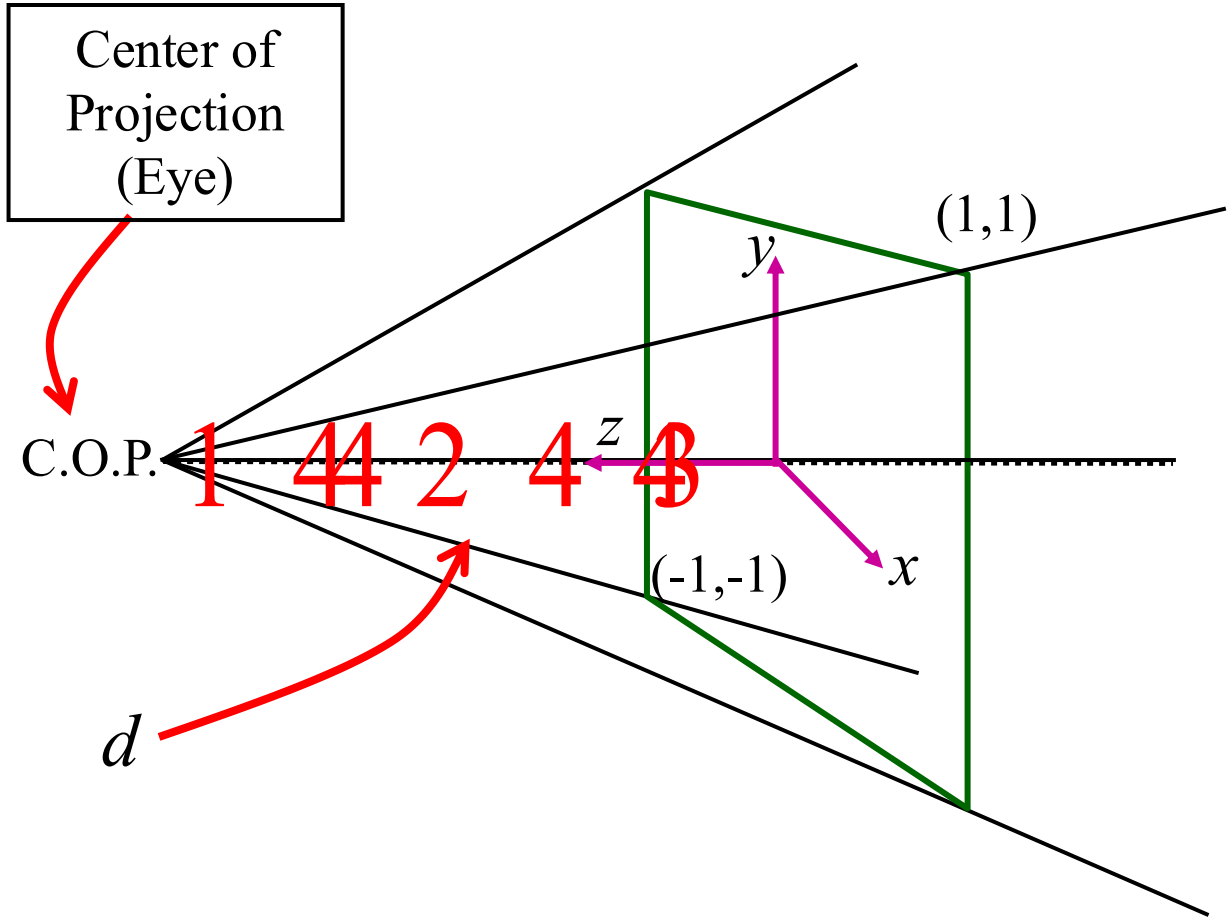
1 4 4 2 4 4 3

make origin COP

Flips the direction of the y-axis

n_x, n_y: width and height of the screen (measured by the no. of pixels)

Geometry of *Perspective Projection*



Explore more detail: Spaces

- So far, we've discussed the following spaces
  - Object space (local space)
  - World space (global space)
  - Camera space

Camera Space

- Let's say we want to render an image of a chair from a certain camera's point of view
- The chair is placed in world space with matrix **W**
- The camera is placed in world space with matrix **C**
- The following transformation takes vertices from the chair's object space into world space, and then from world space into camera space:

$$\mathbf{v}' = \mathbf{C}^{-1} \cdot \mathbf{W} \cdot \mathbf{v}$$

- Now that we have the object transformed into a space relative to the camera, we can focus on the next step, which is to project this 3D space into a 2D image space

Image Space

- What we really need is a mapping from 3D space into a special “2.5D” *image space*
- For *practical* geometric purposes, it should be thought of as a proper 2D space, but each vertex in this space also has a depth (z coordinate), and so mathematically speaking, it really is a 3D space…
- We will say that the visible portion of the image ranges from -1 to 1 in both x and y, with 0,0 in the center of the image
- The z coordinate will also range from -1 to 1 and will represent the depth (1 being nearest and -1 being farthest)
- Image space is sometimes called *normalized view space* or other things…

View Projections

- So far we know how to transform objects from object space into world space and into camera space
- What we need now is some sort of transformation that takes points in 3D camera space and transforms them into our 2.5D image space
- We will refer to this class of transformations as *view projections* (or just projections)
- Simple *orthographic* view projections can just be treated as one more affine transformation applied after the transformation into camera space
- More complex *perspective* projections require a non-affine transformation followed by an additional division to convert from 4D homogeneous space into image space
- It is also possible to do more elaborate *non-linear projections* to achieve fish-eye lens effects, etc., but these require significant changes to the rendering process, not only in the vertex transformation stage, and so we will not cover them…

Orthographic Projection

- An *orthographic* projection is an affine transformation and so it preserves parallel lines
- An example of an orthographic projection might be an architect's top-down view of a house, or a car designer's side view of a car
- These are very valuable for certain applications, but are not used for generating realistic images

Orthographic Projection

$$\mathbf{v}' = \mathbf{P} \cdot \mathbf{C}^{-1} \cdot \mathbf{W} \cdot \mathbf{v}$$

$$\mathbf{P}_{ortho}(left, right, bottom, top, near, far) =$$

$$\begin{bmatrix} \frac{2}{right - left} & 0 & 0 & -\frac{right + left}{right - left} \\ 0 & \frac{2}{top - bottom} & 0 & -\frac{top + bottom}{top - bottom} \\ 0 & 0 & \frac{2}{far - near} & \frac{far + near}{far - near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Perspective Projection

- With a perspective projection, objects become smaller as they get further from the camera, as one normally gets from a photograph or video image
- The goal of most real-world camera lens makers is to achieve a perfect perspective projection
- Most realistic computer generated images use perspective projections
- A fundamental property of perspective projections is that parallel lines will *not* necessarily remain parallel after the transformation (making them non-affine)

View Volume

- A camera defines some sort of 3D shape in world space that represents the volume viewable by that camera
- For example, a normal perspective camera with a rectangular image describes a sort of infinite pyramid in space
- The tip point of the pyramid is at the camera's origin and the pyramid projects outward in front of the camera into space
- In computer graphics, we typically prevent this pyramid from being infinite by chopping it off at some distance from the camera. We refer to this as the *far clipping plane*
- We also put a limit on the closest range of the pyramid by chopping off the tip. We refer to this as the *near clipping plane*
- A pyramid with the tip cut off is known as a *frustum*. We refer to a standard perspective view volume as a *view frustum*

View Frustum

- In a sense, we can think of this view frustrum as a distorted cube, since it has six faces, each with 4 sides
- If we think of this cube as ranging from -1 to 1 in each dimension *xyz*, we can think of a perspective projection as a mapping from this view frustrum into a *normalized view space*, or *image space*
- We need a way to represent this transformation mathematically

View Frustum

- The view frustum is usually defined by a *field of view* (FOV) and an *aspect ratio*, in addition to the near and far clipping plane distances
- Depending on the convention, the FOV may represent either the horizontal or vertical field of view. The most common convention is that the FOV is the entire vertical viewing angle
- The aspect ratio is the x/y ratio of the final displayed image
- For example, if we are viewing on a TV that is 24” x 18”, the aspect ratio would be 24/18 or 4/3
- Older style TV’s have a 4/3 aspect ratio, while modern wide screen TV’s use a 16/9 aspect. These sizes are common for computer monitors as well

Perspective Matrix

$$\mathbf{v}'_{(4D)} = \mathbf{P} \cdot \mathbf{C}^{-1} \cdot \mathbf{W} \cdot \mathbf{v}$$

$$\mathbf{P}_{persp}(fov, aspect, near, far) =$$

$$\begin{bmatrix} \frac{\tan(fov/2)}{aspect} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(fov/2)} & 0 & 0 \\ 0 & 0 & \frac{near + far}{near - far} & \frac{2 \cdot near \cdot far}{near - far} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Perspective Matrix

- If we look at the perspective matrix, we see that it doesn't have [0 0 0 1] on the bottom row
- This means that when we transform a 3D position vector [v_x v_y v_z 1], we will not necessarily end up with a 1 in the 4th component of the result vector
- Instead, we end up with a true 4D vector [v_x' v_y' v_z' v_w']
- The final step of perspective projection is to map this 4D vector back into the 3D w=1 subspace:

$$\begin{bmatrix} v_x & v_y & v_z & v_w \end{bmatrix} \Rightarrow \begin{bmatrix} \frac{v_x}{v_w} & \frac{v_y}{v_w} & \frac{v_z}{v_w} \end{bmatrix}$$

Perspective Transformations

- It is important to make sure that straight lines in 3D space map to straight lines in image space
- This is easy for x and y, but because of the division, it is possible to have nonlinearities in z
- The perspective transformation shown in the previous slide assures a straight line mapping, but there are other perspective transformations possible that don’t preserve this

Viewports

- The final transformation takes points from the -1…1 image space and maps them to an actual rectangle of pixels (device coordinates)
- We can define our final device mapping from normalized view space into an actual rectangular viewport as:

$$\mathbf{D}(x_0, x_1, y_0, y_1) = \begin{bmatrix} (x_1 - x_0)/2 & 0 & 0 & x_0 + (x_1 - x_0)/2 \\ 0 & (y_1 - y_0)/2 & 0 & y_0 + (y_1 - y_0)/2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- The depth value is usually mapped to a 32 bit fixed point value ranging from 0 (near) to 0xffffffff (far)